RK3572 Android 16.0 SDK开发指南

文件标识：RK-KF-YF-E73

发布版本：V1.1.0

日期：2026-06-10

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

产品版本

芯片名称	内核版本
RK3572	kernel-6.12

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V0.0.1	吴良清	2026-03-02	发布RK3572的RKR0.1版本SDK，该版本仅用于硬件调试，不能用于量产
V1.0.0	吴良清	2026-04-16	发布RK3572的RKR4版本
V1.1.0	吴良清	2026-06-10	更新RKR5版本，支持RK3572S，内核单独编译方式修改

文档问题反馈：wlq@rock-chips.com

目录

RK3572 Android 16.0 SDK开发指南

1. RK3572 Android 16.0 SDK支持芯片
2. RK3572 Android 16.0 SDK代码下载编译
   - 2.1 代码下载
     - 2.1.1 下载地址
     - 2.1.2 服务器镜像下载
   - 2.2 搭建自己的repo代码服务器
     - 2.2.1 环境
     - 2.2.2 gitolite搭建
       - 2.2.2.1 服务器端操作
       - 2.2.2.2 客户端操作
     - 2.2.3 repo镜像搭建
       - 2.2.3.1 服务器端操作
       - 2.2.3.2 客户端操作
     - 2.2.4 客户端操作（本地）
   - 2.3 代码管理
     - 2.3.1 切换自己的代码分支
     - 2.3.2 代码修改提交
     - 2.3.3 同步RK的代码
   - 2.4 kernel代码路径说明
   - 2.5 代码编译
     - 2.5.1 设置环境变量
     - 2.5.2 选择目标 lunch
     - 2.5.3 一键编译命令
   - 2.6 各个平台编译命令汇总
   - 2.7 Generic Kernel Image (GKI)
     - 2.7.1 其他编译说明
       - 2.7.1.1 Android16.0不能直接烧写kernel.img和resource.img
       - 2.7.1.2 单独编译kernel生成boot.img
   - 2.8 固件烧写
     - 2.8.1 固件烧写工具
     - 2.8.2 固件说明
   - 2.9 fastboot烧写动态分区
3. 使用DTBO功能
4. 修改fstab文件
5. 修改parameter.txt
6. Android常用配置
   - 6.1 新建产品lunch
7. Kernel dts说明
   - 7.1 新建产品dts
   - 7.2 DTS 结构说明（RK3572 & RK3572S）
8. 补丁发布
9. 文档说明
   - 9.1 外设支持列表
   - 9.2 Camera IQ Tool文档
   - 9.3 rknn-toolkit2开发SDK和文档
   - 9.4 RKDocs文档说明
10. 工具使用
    - 10.1 StressTest
      - 10.1.1 模块相关
      - 10.1.2 非模块相关
    - 10.2 PCBA测试工具
    - 10.3 DeviceTest
    - 10.4 USB驱动
    - 10.5 开发烧写工具

10.5.1 Windows版本
10.5.2 Linux版本
10.6 SD升级启动制作工具
10.7 写号工具
10.8 DDR焊接测试工具
10.9 efuse烧写工具
10.10 efuse/otp签名工具
10.11 工厂生产固件烧写工具
10.12 userdata分区数据预置工具
10.13 Camera IQ Tool
11. 系统调试
11.1 ADB工具
11.2 概述
11.2.1 USB adb使用说明
11.3 ADB常用命令详解
11.4 Logcat工具
11.4.1 Logcat命令使用
11.5 常用的日志过滤方式
11.6 Procrank工具
11.6.1 使用procrank
11.6.2 检索指定内容信息
11.6.3 跟踪进程内存状态
11.7 Dumpsys工具
11.7.1 使用Dumpsys
11.8 Last log 开启
11.9 FIQ模式
12. 常见问题
12.1 当前kernel和u-boot版本？
12.2 如何获取当前SDK对应的RK release版本
12.3 如何确认本地SDK已经完整更新RK发布的SDK状态
12.4 uboot和kernel阶段logo图片替换
12.5 如何修改Android系统仅支持64位系统
12.6 关机充电和低电预充
12.7 Uboot阶段充电图片打包和替换
12.8 HDMI IN配置
12.9 RM310 4G配置
12.10 Wi-Fi休眠策略配置
12.11 Recovery旋转配置
12.12 Android Surface旋转
12.13 替换 AOSP 部分源代码的 remote
12.14 Data区读写速率的优化
12.15 userdata区文件系统换为EXT4
12.16 修改开关机动画和开关机铃声
12.17 APP设置性能模式
12.18 GPU相关问题排查方法
12.19 OTP和efuse说明
12.20 代码中如何判断设备的OTP/EFUSE是否已经烧写
12.21 开关selinux
12.22 开机弹出”Android系统出现问题”警告
12.23 如何打开设置中以太网的设置项
12.24 关于AVB和security boot的操作
12.25 IO命令无法使用
12.26 SN号的命名规则
12.27 Kernel编译报LZ4的错误
12.28 Android Samba功能
12.29 NFS启动
13. 多屏异显异触
14. 多屏异声

15. 附录 A 编译开发环境搭建 Compiling and development environment setup
    15.1 Initializing a Build Environment
    15.2 Choosing a Branch
    15.3 Setting up a Linux build environment
    15.4 Installing the JDK
    15.5 Configuring USB Access
16. 附录 B SSH公钥操作说明 SSH public key operation instruction
    16.1 附录 B-1 SSH公钥生成 SSH public key generation
    16.2 附录 B-2 使用key-chain管理密钥 Use key-chain to manage the key
    16.3 附录 B-3 多台机器使用相同ssh公钥 Multiple devices use the same ssh public key
    16.4 附录 B-4 一台机器切换不同ssh公钥 Switch different ssh public keys on one device
    16.5 附录 B-5 密钥权限管理 Key authority management
    16.6 附录 B-6 Git权限申请说明 Git authority application instruction

1. RK3572 Android 16.0 SDK支持芯片

芯片平台	是否支持	SDK版本
RK3572	支持	RKR0.1
RK3572	支持	RKR4
RK3572	支持	RKR5
RK3572S	支持	RKR5

2. RK3572 Android 16.0 SDK代码下载编译

2.1 代码下载

2.1.1 下载地址

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -b rk3572 -m
Android16.xml
```

如果申请的Express的权限，那么下载地址如下：

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -b rk3572 -m
Android16_Express.xml
```

2.1.2 服务器镜像下载

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -b rk3572 -m
Android16.xml --mirror
```

如果申请的Express的权限，那么下载地址如下：

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/Android_16/manifests -b rk3572 -m
Android16_Express.xml --mirror
```

注，repo是google用Python脚本写的调用git的一个脚本，主要是用来下载、管理Android项目的软件仓库，其下载地址如下：

```
git clone https://gerrit.rock-chips.com:8443/repo-release/tools/repo
```

为方便客户快速获取SDK源码，瑞芯微技术窗口通常会提供对应版本的SDK初始压缩包。以`Rockchip_Android16.0_SDK_RELEASE.tar.gz.*`为例，拷贝到该初始化包后，通过如下命令可检出源码：

```
mkdir rk3572_Android16.0_SDK_RELEASE
cat rk3572_Android16.0_SDK_RELEASE.tar.gz* | tar -zx -C
rk3572_Android16.0_SDK_RELEASE
cd rk3572_Android16.0_SDK_RELEASE
.repo/repo/repo sync -l
.repo/repo/repo sync -c
```

2.2 搭建自己的repo代码服务器

2.2.1 环境

安装`openssh-server`用于远程登录，`git`用于管理工程，`keychain`用于公私钥管理工具

```
sudo apt-get install openssh-server git keychain
```

2.2.2 gitolite搭建

2.2.2.1 服务器端操作

（以服务器地址为例进行说明）

1. 创建git账户：

```
sudo adduser --system --shell /bin/bash --group git
sudo passwd git
```

2. 以“git”账户登录服务器
3. 确保“~/.ssh/authorized_keys”为空或者不存在
4. 拷贝服务器管理员的公钥到“~/YourName.pub”
5. 下载gitolite源码

```
git clone https://github.com/sitaramc/gitolite.git
```

6. 在git用户目录下创建bin目录

```
mkdir -p ~/bin
```

7. 执行下列命令安装gitolite，不同版本安装方法不同，请参考源码中的文档：

```
gitolite/install -to ~/bin
```

8. 设置管理员

```
~/bin/gitolite setup -pk YourName.pub
```

2.2.2.2 客户端操作

1. 克隆服务器的gitolite管理仓库：

```
git clone 服务器地址/gitolite-admin.git
```

2. 添加用户公钥到gitolite目录下

```
cp username.pub keydir/username.pub
```

3. 添加管理员用户

```
vi conf/gitolite.conf
@admin = admin1 admin2 admin3
repo gitolite-admin
RW+     =   @admin
```

2.2.3 repo镜像搭建

2.2.3.1 服务器端操作

1. 用git账号登入服务器
2. 在根目录下载repo工具

```
git clone https://gerrit.rock-chips.com:8443/repo-release/tools/repo
```

3. 新建RK_Android16_mirror目录

```
mkdir RK_Android16_mirror
```

4. 进入 RK_Android16_mirror目录

```
cd RK_Android16_mirror
```

5. 下载RK Android16 SDK镜像

```
repo init --repo-url https://gerrit.rock-chips.com:8443/repo-release/tools/repo
-u https://gerrit.rock-chips.com:8443/android_a16/manifests -b rk3572 -m
Android16.xml --mirror
```

6. 创建仓库组权限

```
.repo/repo/repo list -n > android_a16.conf
sed -i 's/^/@android_t = RK_Android16_mirror\/&/g' android_a16.conf
```

2.2.3.2 客户端操作

1. 将服务器端的android_a16.conf拷贝到客户端的`gitolite-admin/conf/`下
2. 添加组权限

```
vi conf/android_a16.conf
@usergroup = user1 user2 user3
repo @android_a16
R   = @usergroup
RW+ = @admin
```

```
vi conf/gitolite.conf
include "android_a16.conf"
```

3. 新建自己的manifests仓库

```
vi conf/android_a16.conf
@android_a16 = Android_T/manifests_xxx
```

2.2.4 客户端操作（本地）

1. 在客户端下载manifests_xxx仓库

在其他客户端电脑上下载manifests_xxx.git仓库

```
git clone 服务器地址/android_a16/manifests_xxx.git
```

2. 在客户端下载原始manifests仓库

```
git clone 服务器地址/android_a16/manifests.git
```

3. 提交manifest.xml文件到新建的manifest_xxx仓库中

将原始manifests下面的文件拷贝到manifests_xxx内

```
cd manifests_xxx
cp -rf manifests/*.xml manifests_xxx/
```

查看拷贝文件

```
git status

 ├── Android16_Express.xml
├── Android16.xml
├── default_remote.xml
├── default.xml
├── include
```

```
│   ├── partner_gms_express.xml
│   ├── partner_modules_express.xml
│   ├── rk3572_ebook_repository.xml
│   ├── rk3572_repository.xml
│   ├── rk_checkout_repository.xml
│   ├── rk_modules_repository.xml
│   ├── rk_system_core.xml
│   └── ssi_core.xml
├── remote.xml
├── remove_16.xml
├── remove_unused_backup.xml
└── remove_unused.xml
```

本地提交

```
git add -A
git commit -m "init xxx"
```

push到远程分支

```
git push origin master:master
```

4. 创建自己的代码下载链接

在根目录下下载repo工具

```
git clone https://gerrit.rock-chips.com:8443/repo-release/tools/repo
```

按以上步骤操作后，自己的代码下载链接如下

```
mkdir Android16
cd Android16
~/repo/repo init -u 服务器地址/android_a16/manifests_xxx.git -m Android16.xml
```

其中：

`服务器地址` 是你的服务器端地址

通过以上步骤就可以完成自己的repo服务器搭建了，可以把自己的代码服务器链接分享给同事们一起工作了。

2.3 代码管理

通过以上步骤搭建代码服务器后大部分代码仓库都使用RK默认的分支，如果有仓库需要修改自己的代码，可以参考下面的步骤进行操作。

2.3.1 切换自己的代码分支

1. 进入需要修改的代码仓库，以kernel目录为例进行说明

```
cd kernel-6.12
```

2. 切换一个本地分支

```
git checkout remotes/m/master -b xxx_branch
```

3. push xxx_branch分支到远程服务器

```
git push rk29 xxx_branch:xxx_branch
```

其中`rk29`是remote 可以直接tab键自动补全

4. 进入.repo/manifests目录修改manifest里面指定的分支

进入.repo/manifests目录通过grep kernel可以找到kernel仓库对应的manifest的位置

```
cd .repo/manifests
```

```
--- a/include/rk_modules_repository.xml
+++ b/include/rk_modules_repository.xml
@@ -10,7 +10,7 @@
   <project path="hardware/rockchip/libgraphicpolicy"
name="rk/hardware/rk29/libgraphicpolicy" remote="rk"
revision="refs/tags/android-16.0-mid-rkr1" />
   <project path="hardware/rockchip/libhwjpeg" name="rk/hardware/rk29/libhwjpeg"
remote="rk" revision="refs/tags/android-16.0-mid-rkr5"/>
   <project path="u-boot" name="rk/u-boot" remote="rk"
revision="refs/tags/android-16.0-mid-rkr5"/>

-  <project path="kernel" name="rk/kernel" remote="rk29"
revision="refs/tags/android-16.0-mid-rkr5"/>

+  <project path="kernel" name="rk/kernel" remote="rk29" revision="xxx_branch"/>

   <project path="bootable/recovery/rkupdate"
name="platform/bootable/recovery/rk_update" remote="rk"
revision="refs/tags/android-16.0-mid-rkr5"/>
   <project path="bootable/recovery/rkutility"
name="platform/bootable/recovery/rk_utility" remote="rk"
revision="refs/tags/android-16.0-mid-rkr5"/>
```

5. 提交修改的manifest到远程分支

```
git add include/rk_modules_repository.xml
git commit -m "change kernel branch on xxx_branch"
git push origin default:master
```

提交manifests仓库后，其他同事就可以同步到你们自己的分支的kernel代码了。

2.3.2 代码修改提交

按上面步骤切换完分支后就可以在自己分支上提交自己的修改了，提交直接push到xxx_branch分支上面。

2.3.3 同步RK的代码

1. 同步RK代码需要在服务器端进行sync操作

```
cd RK_Android16_mirror
```

```
.repo/repo/repo sync -c
```

2. 客户端合并RK对manifests的修改

- 下载RK原始manifests仓库

```
git clone 服务器地址/username/manifests.git
```

使用对比工具对比manifests（RK原始）和manifests_xxx(自己的)，将RK修改的差异部分合并到自己的仓库中（主要修改tag，增加删除仓库等）。

- 对比确认后将修改push到manifests_xxx上。

这步也可以确认自己修改了哪些仓库，在下一步中将进行修改仓库的合并。

3. 有自己切分支的目录需要手动把RK的修改merge到自己的分支上面

以kernel为例：

- 查看当前指向的远程分支

```
user@host:~/home1/test2/kernel-6.12$ git branch -av
* android-11.0-mid-rkr7   0bde59fad73a ARM: configs: rockchip_defconfig enable
ION_CMA_HEAP
  xxx_branch              0bde59fad73a ARM: configs: rockchip_defconfig enable
ION_CMA_HEAP
  remotes/m/master        -> rk29/xxx_branch
  remotes/rk29/xxx_branch 0bde59fad73a ARM: configs: rockchip_defconfig enable
ION_CMA_HEAP
```

可以看到当前指向的是：`remotes/m/master        -> rk29/xxx_branch`

- 创建本地分支（从自己的远程分支上切）

```
git checkout remotes/m/xxx_branch -b local_xxx_branch
```

- 确认当前RK发布的最新TAG

```
user@host:~/home1/test2/kernel$ git tag | grep rkr
android-10.0-mid-rkr1
android-10.0-mid-rkr10
android-10.0-mid-rkr11
android-10.0-mid-rkr13
android-10.0-mid-rkr2
android-10.0-mid-rkr3
android-10.0-mid-rkr4
android-10.0-mid-rkr5
android-10.0-mid-rkr5
android-10.0-mid-rkr7
```

```
android-10.0-mid-rkr8
android-10.0-mid-rkr9
android-11.0-ebook-rkr1
android-11.0-ebook-rkr2
android-11.0-ebook-rkr3
android-11.0-ebook-rkr4
android-11.0-ebook-rkr5
android-11.0-ebook-rkr6
android-11.0-mid-rkr1
android-11.0-mid-rkr2
android-11.0-mid-rkr3
android-11.0-mid-rkr4
android-11.0-mid-rkr4.1
android-11.0-mid-rkr5
android-11.0-mid-rkr5
android-11.0-mid-rkr7
android-11.0-mid-rkr7-prev
android-11.0-mid-rkr8
android-16.0-mid-rkr0.1
```

可以看到当前最新的Android16的tag是`android-16.0-mid-rkr0.1`

- 合并`android-16.0-mid-rkr0.1`到本地分支

```
git merge android-16.0-mid-rkr0.1
```

查看是否有冲突，如果有冲突先解决冲突，没有冲突再执行下一步

- push合并完的代码到远程分支

```
git push rk29 local_xxx_branch:xxx_branch
```

- 其他切分的目录都按这个方式进行合并提交即可

2.4 kernel代码路径说明

Android 16 SDK 使用的 kernel 版本为 **kernel-6.12**。

RK3572 的 kernel 配置文件采用分层结构：

配置文件	位置	说明
`rockchip_defconfig`	`kernel-6.12/arch/arm64/configs/rockchip_defconfig`	Rockchip 芯片基础 defconfig
`android-16.config`	`kernel-6.12/kernel/configs/android-16.config`	Android 通用配置
`rk3572.config`	`kernel-6.12/arch/arm64/configs/rk3572.config`	RK3572 芯片级配置

编译时 config 组合顺序： `rockchip_defconfig android-16.config <芯片config>`

芯片平台	芯片config
RK3572	`rk3572.config`
RK3572S	`rk3572.config`（与RK3572共用）
RK3576	`rk3576.config`

Android16支持6.12 版本的kernel，kernel源码在工程中kernel-6.12目录下。

2.5 代码编译

2.5.1 设置环境变量

使用 envsetup.sh 脚本初始化环境：

```
source build/envsetup.sh
```

2.5.2 选择目标 lunch

在编译 Android 之前，需要确定一个编译目标（target）。编译目标反映了你所编译的平台。使用 lunch 命令加上一个表示目标的字符串来确定你的编译目标。以下是一个详细的步骤和示例，帮助你完成这个过程。

```
lunch rk3572_a16-bp4a-userdebug
```

表示目标的字符串格式如下:

```
lunch product_name-release-build_variant
```

这个字符串的组成部分如下：

- product_name 是你想要编译的产品的名称，例如 rk3572_a16。你可以根据自己的设备格式来命名特定的 product_name。
- release 设置为 bp4a，bp4a代表目前稳定版本的配置，可用生产。配置细节在build/release/aconfig/bp4a获取，随着Android版本更新，稳定版本的配置也会更新变化，需要关注build/release/aconfig/中的配置变化情况，选择最新的配置用于量产。trunk_staging代表主线的release配置，可以用于Android新功能的调试，配置细节在build/release/aconfig/trunk_staging获取。
- 字符串中的 build_variant 部分可以是以下表格中的三个值之一：

构建类型	使用情况
user	权限受限；适用于生产环境
userdebug	与“user”类似，但具有 root 权限和调试功能；是进行调试时的首选编译类型
eng	具有额外调试工具的开发配置

其他Lunch项说明

lunch项	适用芯片	其他说明
rk3572_a16-bp4a-user	RK3572	适用于Android16的产品，适配RK3572开发板硬件，默认对应的dts文件是rk3572-evb1-v10.dts，编译的是user版本，生产时使用，Android系统仅支持64位
rk3572_a16-bp4a-userdebug	RK3572	适用于Android16的产品，硬件适配RK3572开发板，默认对应的dts文件是rk3572-evb1-v10.dts，编译的是userdebug版本，开发调试时使用，Android系统仅支持64位
rk3572_ebook-bp4a-user	RK3572	适用于Android16的电子书产品，硬件适配RK3572开发板，默认对应的dts文件是rk3572-ebook-a3-v10.dts，编译的是user版本，生产时使用，Android系统支持64位
rk3572_ebook-bp4a-userdebug	RK3572	适用于Android16的电子书产品，硬件适配RK3572开发板，默认对应的dts文件是rk3572-ebook-a3-v10.dts，编译的是userdebug版本，开发调试时使用，Android系统支持64位
rk3572s_a16-bp4a-user	RK3572S	适用于Android16的产品，适配RK3572S芯片，默认对应的dts文件是rk3572s-evb1-v10.dts，编译的是user版本，生产时使用
rk3572s_a16-bp4a-userdebug	RK3572S	适用于Android16的产品，适配RK3572S芯片，默认对应的dts文件是rk3572s-evb1-v10.dts，编译的是userdebug版本，开发调试时使用

2.5.3 一键编译命令

```
./build.sh -UKAup
( WHERE: -U = build uboot
       -C = build kernel with Clang
       -K = build kernel
       -A = build android
       -p = will build packaging in IMAGE
       -o = build OTA package
       -u = build update.img
       -v = build android with 'user' or 'userdebug'
       -d = build kernel dts name
       -V = build version
       -J = build jobs
 )

============================================================
```

```
请注意使用一键编译命令之前需要设置环境变量，选择好自己需要编译的平台，举例：
source build/envsetup.sh
lunch rk3572_a16-bp4a-userdebug
============================================================
```

2.6 各个平台编译命令汇总

Soc	类型	参考机型	Android	一键编译	kernel编译	uboot编译
RK3572	开发板EVB1	rk3572-evb1-v10	build/envsetup.sh;lunch rk3572_a16-bp4a-userdebug	./build.sh -AUCKu	./build.sh -K	./build.sh -U
RK3572	电子书	rk3572-ebook-a3-v10	build/envsetup.sh;lunch rk3572_ebook-bp4a-userdebug	./build.sh -AUCKu	./build.sh -K	./build.sh -U
RK3572S	开发板EVB1	rk3572s-evb1-v10	build/envsetup.sh;lunch rk3572s_a16-bp4a-userdebug	./build.sh -AUCKu	./build.sh -K	./build.sh -U

2.7 Generic Kernel Image (GKI)

Android16 通过 GMS 和 EDLA 认证的产品都强制要求 kernel 使用 GKI。RK3572 SDK 默认没有开启 GKI，如需开启 GKI 功能可以按如下修改：

```
user@host:~/b0_A16_bringup/device/rockchip/rk3572$ git diff
diff --git a/rk3572_a16/BoardConfig.mk b/rk3572_a16/BoardConfig.mk
index eb52fb6..f59df62 100755
--- a/rk3572_a16/BoardConfig.mk
+++ b/rk3572_a16/BoardConfig.mk
@@ -14,7 +14,7 @@
 # limitations under the License.
 #
 BUILD_WITH_GO_OPT := false
-BOARD_BUILD_GKI := false
+BOARD_BUILD_GKI := true

 BOARD_GRAVITY_SENSOR_SUPPORT := true
```

`BOARD_BUILD_GKI := true` 后会自动开启 AB 分区功能。关于 GKI 的 kernel 编译、ko 更新等具体说明请参考文档：

```
RKDocs/android/Rockchip_Developer_Guide_Android14_GKI_CN.pdf
```

2.7.1 其他编译说明

2.7.1.1 Android16.0不能直接烧写kernel.img和resource.img

以下编译仅适用于非GKI，GKI的请参考文档RKDocs/android/《Rockchip_Developer_Guide_Android14_GKI_CN》

Android16.0的kernel.img和resource.img包含在boot.img中，需要使用build.sh -AK 命令来编译kernel，编译后烧写rockdev下面的boot.img。这个过程会重新编译Android，所以编译时间会比较长，建议用下面单独编译kernel的方式来编译。

2.7.1.2 单独编译kernel生成boot.img

> 注意： kernel-6.12 的编译工具需要使用 rust，与之前的内核版本不同，建议使用下面的 mkboot.sh 脚本方式编译。非 GKI 和 GKI 均可参考以下方式编译。

编译的原理：在kernel目录下将编译生成的`kernel.img`和`resource.img`替换到旧的`boot.img`中。
以`RK3572`样机为例，编译时替换对应的boot.img及dts：
其中`BOOT_IMG=../rockdev/Image-rk3572_a16/boot.img`这里指定的是旧的boot.img的路径，需要根据实际路径修改。

建议将下面的编译脚本保存为`mkboot.sh`，放在`kernel-6.12`目录下，然后执行编译。

```
#!/bin/bash
# mkboot.sh - 单独编译kernel生成boot.img
# 用法: ./mkboot.sh <dts名称> <输出boot.img路径> [芯片config]
# 示例: ./mkboot.sh rk3572-evb1-v10 boot_sample.img rk3572.config

export PATH=$PWD/../prebuilts/clang/host/linux-x86/clang-r574158/bin:$PATH
export PATH=$PWD/../prebuilts/rust/linux-x86/1.88.0/bin:$PATH
CLANG_PATH=$PWD/../prebuilts/clang/host/linux-x86/clang-r574158
RUST_PATH=$PWD/../prebuilts/rust/linux-x86/1.88.0

ADDON_ARGS="CROSS_COMPILE=aarch64-linux-gnu- LLVM=1 LLVM_IAS=1
KBUILD_GENDWARFKSYMS_STABLE=1 ARCH=arm64"
KERNEL_BUILD_TOOL="LD_LIBRARY_PATH=$PWD/../prebuilts/kernel-build-tools/linux-
x86/lib64 PATH=$PWD/../prebuilts/kernel-build-tools/linux-
x86/bin:$PWD/$CLANG_PATH/bin:$PWD/$RUST_PATH/bin:$PATH HOSTCFLAGS=-
I$PWD/../prebuilts/kernel-build-tools/linux-x86/include HOSTLDFLAGS=-
L$PWD/../prebuilts/kernel-build-tools/linux-x86/lib64 LIBCLANG_PATH=\$(dirname
\$(which clang))/../lib BINDGEN=$PWD/../prebuilts/clang-tools/linux-
x86/bin/bindgen"

make $KERNEL_BUILD_TOOL $ADDON_ARGS rockchip_defconfig android-16.config \$3 &&
make $KERNEL_BUILD_TOOL $ADDON_ARGS BOOT_IMG=\$2 \$1.img -j32
```

编译命令示例：

RK3572（EVB1）：

```
cd kernel-6.12
./mkboot.sh rk3572-evb1-v10 ../rockdev/Image-rk3572_a16/boot.img rk3572.config
```

RK3572S（EVB1）：

```
cd kernel-6.12
./mkboot.sh rk3572s-evb1-v10 ../rockdev/Image-rk3572s_a16/boot.img rk3572.config
```

> 注意：不同芯片要添加不同的config文件，一般是芯片名称+.config，比如：`rk3572.config`、`rk3576.config`、`rk3588.config`。

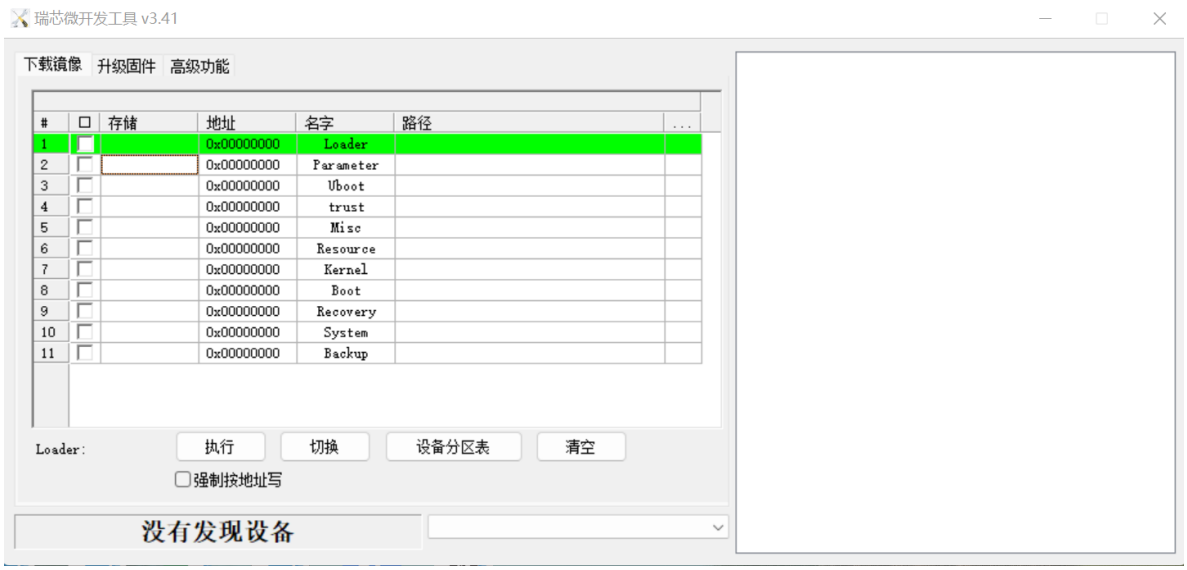
编译后可以直接烧写kernel-6.12目录下生成的boot.img到机器的boot位置，烧写时请先加载分区表（**parameter.txt**），以免烧写位置错误。

2.8 固件烧写

2.8.1 固件烧写工具

RK3572 Android16的USB驱动DriverAssitant需要更新到V5.14版本，可以参考下面的工具章节进行更新。
Windows烧写工具：（工具是时刻更新，请及时同步更新）
RK3572平台windows固件烧写工具必须使用V3.41及以上版本

```
RKTools\windows\AndroidTool/RKDevTool_v3.41_for_window
```



Linux烧写工具：
RK3572平台Linux固件烧写工具必须使用V2.55及以上版本

```
RKTools/linux/Linux_Upgrade_Tool/upgrade_tool_v2.55_for_linux
```

在下文工具说明章节有详细说明

2.8.2 固件说明

固件	说明
boot.img	包含ramdisk、kernel、dtb
boot-debug.img	与boot.img的差别是user固件可以烧写这个boot.img进行root权限操作
dtbo.img	Device Tree Overlays 参考下面的dtbo章节说明
config.cfg	烧写工具的配置文件，可以直接导入烧写工具显示需要烧写的选项
MiniLoaderAll.bin	包含一级loader
misc.img	包含recovery-wipe开机标识信息，烧写后会进行recovery
parameter.txt	包含分区信息
pcba_small_misc.img	包含pcba开机标识信息，烧写后会进入简易版pcba模式
pcba_whole_misc.img	包含pcba开机标识信息，烧写后会进入完整版pcba模式
recovery.img	包含recovery-ramdisk、kernel、dtb
super.img	包含odm、product、vendor、system、system_ext分区内容
trust.img	包含BL31、BL32 RK3566/RK3568没有生成这个固件，不需要烧写
uboot.img	包含uboot固件
vbmeta.img	包含avb校验信息，用于AVB校验
update.img	包含以上需要烧写的img文件，可以用于工具直接烧写整个固件包

2.9 fastboot烧写动态分区

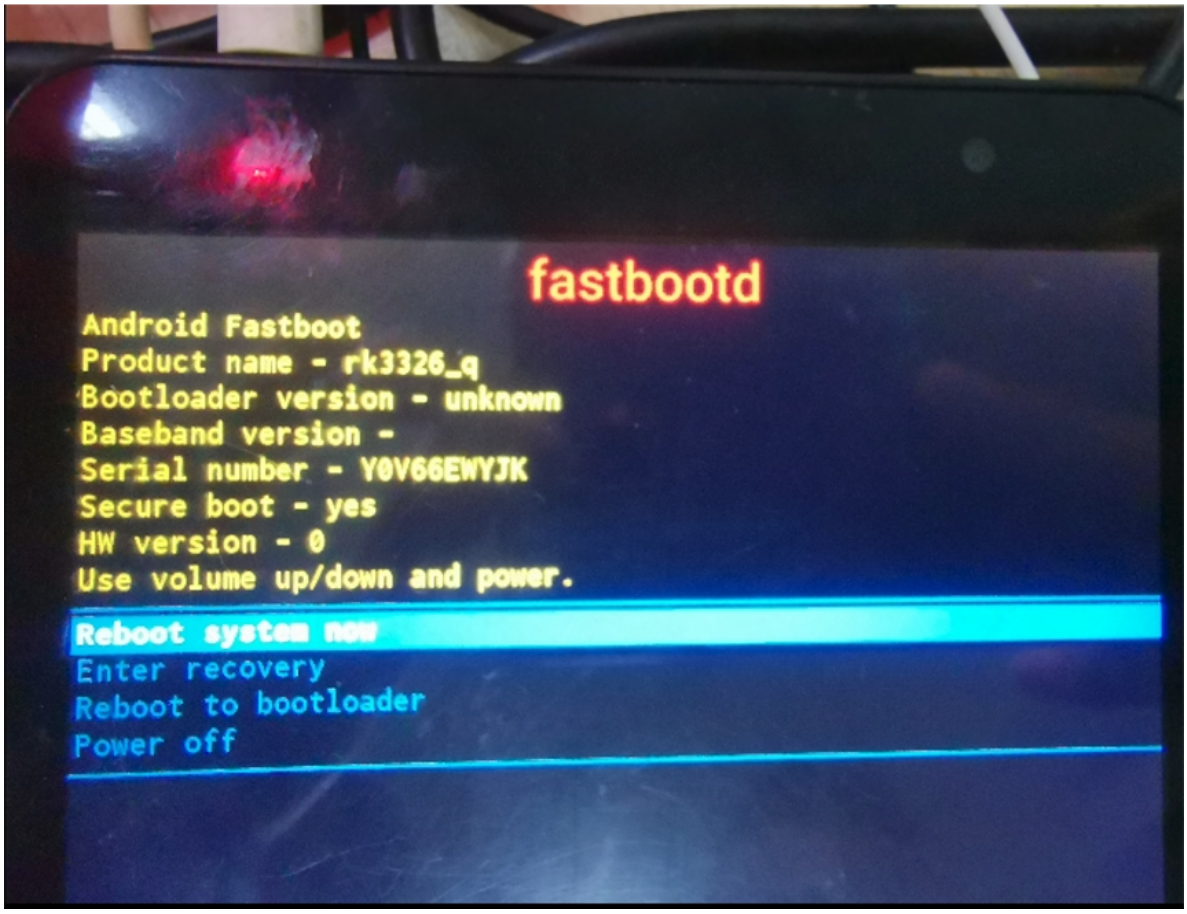
新设备支持动态分区，已经移除了system/vendor/odm/product/system_ext分区，请烧写super.img，单独烧写system/vendor/odm等（可以在out下面找到对应img文件）可以用`fastbootd`，要求adb和fastboot版本均为最新，SDK提供了编译好的工具包：

```
RKTools/linux/Linux_adb_fastboot (Linux_x86版本)
RKTools/windows/adb_fastboot (Windows_x86版本)
```

- 使用命令烧写动态分区：

```
adb reboot fastboot
fastboot flash vendor vendor.img
fastboot flash system system.img
fastboot flash odm odm.img
```

注：进入fastbootd模式后，屏幕上会显示相关设备信息，如图所示：



注：非动态分区使用**fastboot**，请进入**bootloader**:

```
adb reboot bootloader
```

烧写GSI的方法：

- 确认机器解锁后，进入fastbootd，只需要烧写GSI中的system.img及固件中的misc.img，烧写后会进入recovery进行恢复出厂设置。下面附上整个烧写流程：

1. 重启至bootloader，未解锁->机器解锁：

```
adb reboot bootloader
fastboot oem at-unlock-vboot ## 对于烧写过avb公钥的客户，请参考对应的文档解锁。
```

2. 恢复出厂设置，重启至fastbootd：

```
fastboot flash misc misc.img
fastboot reboot fastboot  ## 此时将进入fastbootd
```

3. 开始烧写GSI

```
fastboot delete-logical-partition product  ## (可选)对于分区空间紧张的设备，可以先执行本条命令删除product分区后再烧写GSI
fastboot flash system system.img
fastboot reboot  ## 烧写成功后，重启
```

- 注：也可以使用DSU(Dynamic System Updates)烧写GSI，目前Rockchip平台已经默认支持DSU。由于该功能需要消耗大量内存，不建议1G DDR及以下的设备使用，有关DSU的说明和使用，请参考Android官网：

https://source.android.com/devices/tech/ota/dynamic-system-updates

- 注1：VTS测试时，需要同时烧写编译出的boot-debug.img到boot分区；
- 注2：CTS-ON-GSI测试时则不需要烧boot-debug.img；
- 注3：测试时请使用Google官方发布的，带有-signed结尾的GSI镜像；

3. 使用DTBO功能

Android 10.0及以上支持Device Tree Overlays功能，开发过程体现在需要烧写dtbo.img，用于多个产品间的兼容等。
修改方法：

1. 找到(或指定)模板文件：

```
get_build_var PRODUCT_DTBO_TEMPLATE
```

例如：

```
PRODUCT_DTBO_TEMPLATE := $(LOCAL_PATH)/dt-overlay.in
(device/rockchip/rk3572/rk3572_a16/dt-overlay.in)
```

2. 添加或修改需要的节点：

例如：

```
/dts-v1/;
/plugin/;

&chosen {
    bootargs_ext = "androidboot.boot_devices=${_boot_device}";
};

&firmware_android {
    vbmeta {
        status = "disabled";
    };
```

```
    fstab {
        status = "disabled";
    };
};

&reboot_mode {
    mode-bootloader = <0x5242C309>;
    mode-charge = <0x5242C30B>;
    mode-fastboot = <0x5242C303>;
    mode-loader = <0x5242C301>;
    mode-normal = <0x5242C300>;
    mode-recovery = <0x5242C303>;
};
```

注: 使用dtbo时一定要确保dts中存在alias，否则无法成功overlay

4. 修改fstab文件

1. 找到(或指定)模板文件：

```
get_build_var PRODUCT_FSTAB_TEMPLATE
```

例如：

```
PRODUCT_FSTAB_TEMPLATE := device/rockchip/common/scripts/fstab_tools/fstab.in
```

2. 修改：添加分区挂载、修改swap_zram参数，修改data分区格式等

5. 修改parameter.txt

Android 14添加了生成parameter.txt的工具，支持根据配置参数编译出parameter.txt。如果没有配置模板文件，则会寻找添加修改好的parameter.txt文件。

1. 找到(或指定)模板文件：

```
get_build_var PRODUCT_PARAMETER_TEMPLATE
```

例如：

```
PRODUCT_PARAMETER_TEMPLATE :=
device/rockchip/common/scripts/parameter_tools/parameter.in
```

2. 修改配置分区大小(例如)：

```
BOARD_SUPER_PARTITION_SIZE := 2688548864
BOARD_DTBOIMG_PARTITION_SIZE := xxxx
BOARD_BOOTIMAGE_PARTITION_SIZE := xxxxx
BOARD_CACHEIMAGE_PARTITION_SIZE := xxxx
```

3. 不使用parameter生成工具：

添加一个parameter.txt文件到你的device目录下即可：
例如：device/rockchip/rk3572/rk3572_a16/parameter.txt

4. 仅使用工具生成parameter.txt(例如)：

```
parameter_tools --input
device/rockchip/common/scripts/parameter_tools/parameter.in --firmware-version
16.0 --machine-model rk3572 --manufacturer rockchip --machine rk3572_a16 --
partition-list
uboot_a:4096K,trust_a:4M,misc:4M,dtbo_a:4M,vbmeta_a:4M,boot_a:33554432,backup:30
0M,security:4M,cache:300M,metadata:4096,frp:512K,super:2G --output
parameter_new.txt
```

注: 如果需要大版本OTA升级，请直接使用之前版本的parameter.txt

5. 新加一个分区

以新建baseparameter分区为例进行说明：

- 在产品的BoardConfig.mk中定义：BOARD_WITH_SPECIAL_PARTITIONS

like: BOARD_WITH_SPECIAL_PARTITIONS := baseparameter:1M,logo:16M

```
device/rockchip/rk3572/rk3572_a16/BoardConfig.mk
+++ b/BoardConfig.mk
@@ -494,4 +494,11 @@ ifeq ($(strip $(BOARD_TWRP_ENABLE)), true)
+BOARD_WITH_SPECIAL_PARTITIONS := baseparameter:1M
```

- 在RebuildParameter.mk中添加BOARD_WITH_SPECIAL_PARTITIONS

```
device/rockchip/common/build/rockchip/RebuildParameter.mk
+ifneq ($(strip $(BOARD_WITH_SPECIAL_PARTITIONS)), )
+partition_list := $(partition_list),$(BOARD_WITH_SPECIAL_PARTITIONS)
+endif
```

6. Android常用配置

6.1 新建产品lunch

- 进入device/rockchip/rk3572目录。
- 在device/rockchip/rk3572目录下创建一个名为rk3572_new_a16的新目录。
- 参考device/rockchip/rk3572目录下已有的rk3572_a16产品目录进行创建。你可以首先复制

rk3572_a16目录，然后将其重命名为rk3572_new_a16。

你可以使用以下命令复制目录：

```
cp -r rk3572_a16 rk3572_new_a16
```

- 复制目录后，进入rk3572_new_a16目录。

将rk3572_new_a16目录中的所有文件中的rk3572_a16字符串替换为rk3572_new_a16。你可以使用类似sed的工具或手动编辑文件进行修改。
例如，你可以使用如下的sed命令：

```
sed -i 's/rk3572_a16/rk3572_new_a16/g' *
```

这个命令将会在rk3572_new_a16目录下的所有文件中将rk3572_a16替换为rk3572_new_a16。

- 修改完文件后，你的新产品目录rk3572_new_a16已经可以用于进一步的定制和开发。

7. Kernel dts说明

7.1 新建产品dts

产品新建dts可以根据下表的配置选择对应的dts作为参考。

7.2 DTS 结构说明（RK3572 & RK3572S）

Kernel DTS 源码位于 `kernel-6.12/arch/arm64/boot/dts/rockchip/` 目录下，采用**芯片级 .dtsi → 主板级 .dtsi → 产品级 .dts** 的分层结构。

RK3572 DTS 层次结构（以 EVB1 为例）：

```
rk3572.dtsi                ← 芯片级：定义 CPU、GIC、CRU、GRF、VOP、HDMI、显示子系统等
  ├── rk3572-evb.dtsi      ← 主板级：EVB 公共外设（按键、ADC、I2C 等）
  │     └── rk3572-evb1-v10.dtsi    ← 板级：EVB1 具体配置（音频、摄像头、显示屏等）
  │           └── rk3572-evb1-v10.dts  → 产品 DTS（入口文件）
  │
  ├── rk3572-rk806.dtsi    ← PMIC RK806 配置
  └── rk3572-pinctrl.dtsi ← 引脚复用配置
```

`rk3572-evb1-v10.dts` 的 include 链：

```
#include "rk3572-evb1-v10.dtsi"    // 板级配置
#include "rk3572-android.dtsi"     // Android 系统配置
#include "rk3572-evb1-cam-csi0.dtsi"  // 摄像头配置
```

RK3572S DTS 层次结构：

```
rk3572.dtsi                ← RK3572 芯片级（与 RK3572 共用）
  └── rk3572s.dtsi         ← RK3572S 差异层（删除/修改 RK3572 中不支持的模块）
        └── rk3572s-evb.dtsi     ← RK3572S EVB 主板级
              └── rk3572s-evb1-v10.dtsi  ← RK3572S EVB1 板级
                    └── rk3572s-evb1-v10.dts  → 产品 DTS（入口文件）
```

RK3572S 与 RK3572 的主要差异（`rk3572s.dtsi` 中体现）：

- 删除 Ethernet（gmac0/gmac1）、PCIe1、SATA0/SATA1、NPU
- 删除 CAN0-CAN3、DSMC、EBC、VOPL
- 删除 vp1（仅保留 vp0）
- Display 子系统 ports 仅保留 `<&vop_out>`（删除 vopl）
- PD_NPU 删除 pm_perf/pm_qos 配置
- PD_VOP 的 pm_qos 仅保留 `<&qos_vop_m0>`（删除 m1）

Soc	PMIC	开发板类型	机型	DTS
RK3572	RK806	开发板	RK3572 EVB1	rk3572-evb1-v10.dts
RK3572	RK806	开发板	RK3572 EVB2	rk3572-evb2-v10.dts
RK3572	RK809+RK805	平板样机	RK3572 TABLET	rk3572-tablet-v10.dts
RK3572	RK806	电子书	RK3572 EBook	rk3572-ebook-a3-v10.dts
RK3572S	RK806	开发板	RK3572S EVB1	rk3572s-evb1-v10.dts

8. 补丁发布

在redmine系统上面会不定期发布一些重要的补丁，链接如下：

```
https://redmine.rock-chips.com/projects/rockchip_patch/issues
```

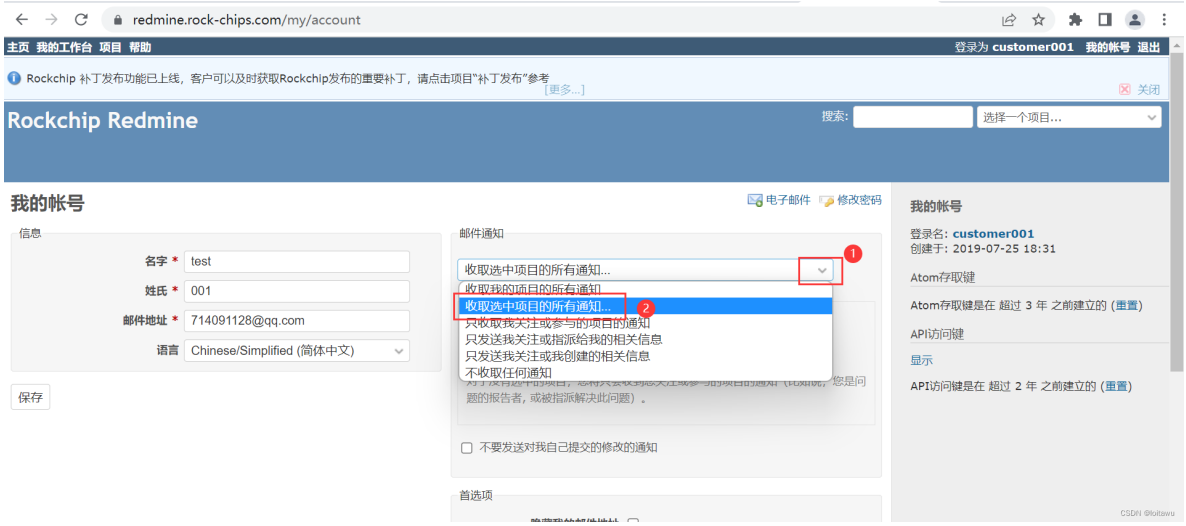
可以通过订阅的方式实时获取补丁发布的邮件通知，订阅方式如下：
第一步　登入redmine系统
使用已经在Rockchip注册的redmine账号登入redmine系统。

第二步 进入我的账号



第三步 选择邮件通知类型

如下图在**邮件通知**中下拉选择**收取选中项目的所有通知**



第四步 选择项目

如下图勾选**补丁发布**项目，并点击**保存**



以上操作即完成补丁发布的订阅。
订阅成功后当Rockchip有补丁发布时即可通过在redmine上面登记的邮箱接收到邮件通知。

9. 文档说明

9.1 外设支持列表

DDR/EMMC/NAND FLASH/WIFI/3G/CAMERA的支持列表实时更新在redmine上，链接如下：

```
https://redmine.rockchip.com.cn/projects/fae/documents
```

9.2 Camera IQ Tool文档

```
external/camera_engine_rkaiq/rkisp2x_tuner/doc/
├── Rockchip_Color_Optimization_Guide_ISP2x_CN_v2.0.0.pdf
├── Rockchip_IQ_Tools_Guide_ISP2x_CN_v2.0.0.pdf
└── Rockchip_Tuning_Guide_ISP21_CN_v2.0.0.pdf
```

9.3 rknn-toolkit2开发SDK和文档

```
hardware/rockchip/rknn-toolkit2/doc/
```

9.4 RKDocs文档说明

```
RKDocs/
├── android
│   ├── Android 异显开发说明.zip
│   ├── audio
│   │   ├── Rockchip_Developer_Guide_Android_3A_Module_CN.pdf
│   │   ├── Rockchip_Developer_Guide_Android_Audio_AIDL_HAL_CN.pdf
│   │   ├── Rockchip_Developer_Guide_Android_EQ_DRC_CN.pdf
│   │   ├── Rockchip_Developer_Guide_Android_Multi_Audio_CN.pdf
│   │   └── Rockchip_Developer_Guide_Automotive_Audio_CN.pdf
│   ├── bt
│   │   └── Rockchip_Introduction_Android9.0_BT_Configuration_CN.pdf
│   ├── patches
│   │   └── root
│   │       ├── Rockchip_Introduction_Android_Root_CN.pdf
│   │       ├── Rockchip_Introduction_Android_Root_EN.pdf
│   │       └── RootChecker.apk
│   ├── project.config
│   ├── Rockchip_Android_GMS_Media_FAQ_CN.pdf
│   ├── Rockchip_Android_Remote_key_Provisioning_Guide.pdf
│   ├── Rockchip_Developer_Guide_AB_System_OTA_from_Android14_to_Android16_CN.pdf
│   ├── Rockchip_Developer_Guide_Android_Optimization_CN.pdf
│   ├── Rockchip_Developer_Guide_Android14_GKI_CN.pdf
│   ├── Rockchip_Developer_Guide_Android14_GKI_EN.pdf
│   ├── Rockchip_Developer_Guide_Android16_SSI_SDK_CN.pdf
│   ├── Rockchip_Developer_Guide_Android16_SSI_SDK_EN.pdf
│   ├── Rockchip_Developer_Guide_Android_AB_System_Upgrading_CN.pdf
│   ├── Rockchip_Developer_Guide_Android_Automotive_CN.pdf
│   ├── Rockchip_Developer_Guide_Android_GRF_OTA_CN.pdf
│   ├── Rockchip_Developer_Guide_Android_Recovery_CN.pdf
│   ├── Rockchip_Developer_Guide_Android_SELinux(Sepolicy)_CN.pdf
│   ├── Rockchip_Developer_Guide_PCBA_Test_Tool_V1.3_CN&EN.pdf
│   ├── Rockchip_Firmware_Upgrade_Failed_Analyze_Method_CN.pdf
│   ├── Rockchip_Introduction_Android_Application_Preinstallation_CN&EN.pdf
│   ├── Rockchip_Introduction_Android_Boot_Video_CN.pdf
│   ├── Rockchip_Introduction_Android_BOX_Display_Framework_Configuration_CN.pdf
│   ├── Rockchip_Introduction_Android_Factory_Reset_Protection_CN&EN.pdf
│   ├── Rockchip_Introduction_Android_Log_System.pdf
│   ├── Rockchip_Introduction_Android_Performance_Mode_CN&EN.pdf
│   ├── Rockchip_Introduction_Android_Power_On_Off_Animation_and_Tone_Customization_CN&EN.pdf
│   ├── Rockchip_Introduction_Android_Samba_CN.pdf
│   ├── Rockchip_Introduction_Android_Widevine_Project_Start_Preparation_CN.pdf
│   ├── Rockchip_Introduction_Box_Media_Application_CN&EN.pdf
│   ├── Rockchip-Parameter-File-Format-Version1.4-CN.pdf
│   ├── Rockchip_User_Guide_Android_Device_ID_Attestation_Flash_CN.pdf
│   ├── Rockchip_User_Guide_Android_GMS_Configuration_CN.pdf
│   ├── Rockchip_User_Guide_Android_GMS_Configuration_EN.pdf
│   ├── Rockchip_User_Guide_Box_FactoryTestTool_V3.0_CN.pdf
│   ├── Rockchip_User_Guide_Dr.G_CN&EN.pdf
│   ├── Rockchip_User_Guide_Magisk_Installation_EN.pdf
│   ├── video
│   │   ├── Rockchip_Android_Introduction_Libhwjpeg_Interface_CN.pdf
│   │   ├── Rockchip_Android_Introduction_Libhwjpeg_Interface_EN.pdf
│   │   ├── Rockchip_Android_Multimedia_FAQ_CN.pdf
```

```
│   │   ├── Rockchip_Android_Multimedia_FAQ_EN.pdf
│   │   ├── Rockchip_Android_Widevine_L1_Guide.pdf
│   │   ├── Rockchip_Developer_Guide_Android_Boot_Video_CN.pdf
│   │   ├── Rockchip_Developer_Guide_Android_Boot_Video_EN.pdf
│   │   ├── Rockchip_Introduction_Android_MediaCodec_Vendor_Parameters_CN.pdf
│   │   ├── Rockchip_Introduction_Android_MediaCodec_Vendor_Parameters_EN.pdf
│   │   └── Rockchip_Introduction_Android_Smart_Encoder_CN.pdf
│   └── wifi
│       ├── Rockchip_Introduction_REALTEK_WIFI_Driver_Porting_CN&EN.pdf
│       └── Rockchip_Introduction_WIFI_Configuration_CN&EN.pdf
└── common
    ├── Audio
    │   ├──
Rockchip_Developer_Guide_Audio_Call_3A_Algorithm_Integration_and_Parameter_Debug
ging_CN.pdf
    │   ├── Rockchip_Developer_Guide_Audio_CN.pdf
    │   ├── Rockchip_Developer_Guide_RK817_RK809_Codec_CN.pdf
    │   └── Rockchip_User_Guide_Android14_Audio_Function_CN.pdf
    ├── camera
    │   ├── common
    │   │   └── Camera_External_FAQ_v1.0.pdf
    │   ├── HAL1
    │   │   ├── README_CN.txt
    │   │   ├── README_EN.txt
    │   │   ├── RK312x_Camera_User_Manual_v1.4(3288&3368).pdf
    │   │   ├── RK_ISP10_Camera_User_Manual_v2.3.pdf
    │   │   ├── RKISPV1_Camera_Module_AVL_v1.7.pdf
    │   │   ├── Rockchip_Camera_AVL_v2.0_Package_20180515.7z
    │   │   ├──
Rockchip_Introduction_RKISPV1_Camera_Driver_Debugging_Method_CN.pdf
    │   │   ├── Rockchip_Introduction_RKISPV1_Camera_FAQ_CN.pdf
    │   │   ├── Rockchip SOFIA 3G-
R_PMB8018(x3_C3230RK)_Camera_Module_AVL_v1.6_20160226.pdf
    │   │   └── Rockchip_Trouble_Shooting_Android_CameraHAL1_CN_EN.pdf
    │   ├── HAL3
    │   │   ├── camera_engine_rkisp_user_manual_v2.2.pdf
    │   │   ├── camera_hal3_user_manual_v2.3.pdf
    │   │   ├── README_CN.txt
    │   │   ├── RKCIF_Driver_User_Manual_v1.0.pdf
    │   │   ├── RKISP1_IQ_Parameters_User_Guide_v1.2.pdf
    │   │   ├── RKISP_Driver_User_Manual_v1.3.pdf
    │   │   ├── Rockchip_Camera_Module_OTP_Calibration_Guide
    │   │   │   └── ISP30
    │   │   │       ├── CN
    │   │   │       │   └── Rockchip Camera Module OTP Calibration Guide_CN.pdf
    │   │   │       └── EN
    │   │   │           └── Rockchip Camera Module OTP Calibration Guide_EN.pdf
    │   │   ├── Rockchip_Color_Optimization_Guide_ISP
    │   │   │   ├── ISP21
    │   │   │   │   └── CN
    │   │   │   │       └──
Rockchip_Color_Optimization_Guide_ISP21_CN_v2.0.1.pdf
    │   │   │   ├── ISP30
    │   │   │   │   ├── CN
    │   │   │   │   │   └──
Rockchip_Color_Optimization_Guide_ISP30_CN_v3.0.2.pdf
    │   │   │   │   └── EN
```

```
    │   │   │   │       └──
Rockchip_Color_Optimization_Guide_ISP30_EN_v3.0.2.pdf
    │   │   │   ├── ISP32-lite
    │   │   │   │   └── CN
    │   │   │   │       └──
Rockchip_Color_Optimization_Guide_ISP32_Lite_CN_v3.1.0.pdf
    │   │   │   └── ISP39
    │   │   │       └── CN
    │   │   │           └──
Rockchip_Color_Optimization_Guide_ISP39_CN_v1.0.0.pdf
    │   │   ├── Rockchip_Developer_Guide_Android_Camera_Driver_CN.pdf
    │   │   ├── Rockchip_Development_Guide_3A_ISP
    │   │   │   ├── ISP30
    │   │   │   │   ├── CN
    │   │   │   │   │   └── Rockchip_Development_Guide_3A_ISP30_v1.1.0.pdf
    │   │   │   │   └── EN
    │   │   │   │       └── Rockchip_Development_Guide_3A_ISP30_EN_v1.1.0.pdf
    │   │   │   └── ISP39
    │   │   │       └── CN
    │   │   │           └── Rockchip_Development_Guide_3A_ISP39.pdf
    │   │   ├── Rockchip_Development_Guide_ISP
    │   │   │   ├── ISP21
    │   │   │   │   └── CN
    │   │   │   │       └── Rockchip_Development_Guide_ISP21_CN_v2.1.0.pdf
    │   │   │   ├── ISP30
    │   │   │   │   ├── CN
    │   │   │   │   │   └── Rockchip_Development_Guide_ISP30_CN_v1.2.5.pdf
    │   │   │   │   └── EN
    │   │   │   │       └── Rockchip_Development_Guide_ISP30_EN_v1.2.5.pdf
    │   │   │   ├── ISP32-lite
    │   │   │   │   └── CN
    │   │   │   │       └── Rockchip_Development_Guide_ISP32_Lite_CN_v1.0.0.pdf
    │   │   │   └── ISP39
    │   │   │       └── CN
    │   │   │           └── Rockchip_Development_Guide_ISP39_CN_alpha.pdf
    │   │   ├── Rockchip_Driver_Guide_VI
    │   │   │   ├── CN
    │   │   │   │   └── Rockchip_Driver_Guide_VI_CN_v1.1.5.pdf
    │   │   │   └── EN
    │   │   │       └── Rockchip_Driver_Guide_VI_EN_v1.1.5.pdf
    │   │   ├── Rockchip_IQ_Tools_Guide_ISP
    │   │   │   ├── ISP21
    │   │   │   │   └── Rockchip_IQ_Tools_Guide_ISP2x_CN_v2.0.3.pdf
    │   │   │   ├── ISP30
    │   │   │   │   └── Rockchip_IQ_Tools_Guide_ISP21_ISP30_CN_v2.0.4.pdf
    │   │   │   └── ISP32-lite
    │   │   │       └── CN
    │   │   │           └── Rockchip_IQ_Tools_Guide_v2.0.7_CN.pdf
    │   │   ├── Rockchip_Trouble_Shooting_Android_GMS_Camera_FAQ_CN.pdf
    │   │   ├── Rockchip_Trouble_Shooting_CameraHAL3_CN.pdf
    │   │   ├── Rockchip_Tuning_Guide_ISP
    │   │   │   ├── ISP21
    │   │   │   │   └── CN
    │   │   │   │       └── Rockchip_Tuning_Guide_ISP21_CN_v2.1.0.pdf
    │   │   │   ├── ISP30
    │   │   │   │   ├── CN
    │   │   │   │   │   └── Rockchip_Tuning_Guide_ISP30_CN_v1.2.0.pdf
    │   │   │   │   └── EN
```

```
    │   │   │   │       └── Rockchip_Tuning_Guide_ISP30_EN_v1.2.0.pdf
    │   │   │   ├── ISP32-lite
    │   │   │   │   └── CN
    │   │   │   │       └── Rockchip_Tuning_Guide_ISP32-lite_CN_v1.0.0.pdf
    │   │   │   └── ISP39
    │   │   │       └── CN
    │   │   │           └── Rockchip_Tuning_Guide_ISP39_CN_v1.0.pdf
    │   │   └── USB_UVC_Integrated_Cameras.pdf
    │   ├── README.txt
    │   └── vehicle
    │       ├──
Rockchip_Android_Fast_Reverse_Image_System_Developer_Guide_CN.pdf
    │       └── Rockchip_Developer_Guide_Automotive_Camera_CN.pdf
    ├── Can
    │   ├── Rockchip_Developer_Guide_Can_CN.pdf
    │   └── Rockchip_Developer_Guide_CAN_FD_CN.pdf
    ├── CLK
    │   ├── Rockchip_Developer_Guide_Clock_CN.pdf
    │   ├── Rockchip_Developer_Guide_Gpio_Output_Clocks_CN.pdf
    │   ├── Rockchip_Developer_Guide_Pll_Ssmod_Clock_CN.pdf
    │   └── Rockchip_RK3399_Developer_Guide_Clock_CN.pdf
    ├── CRYPTO
    │   ├── Rockchip_Developer_Guide_Crypto_HWRNG_CN.pdf
    │   └── Rockchip_Developer_Guide_Crypto_HWRNG_EN.pdf
    ├── DDR
    │   ├── DDR_bandwidth_statistics_tool
    │   │   ├── rk-msch-probe-for-user-32bit
    │   │   └── rk-msch-probe-for-user-64bit
    │   ├── Rockchip_Developer_Guide_DDR_CN.pdf
    │   └── Rockchip_Developer_Guide_DDR_EN.pdf
    ├── debug
    │   ├── Rockchip_Developer_Guide_DS5_CN.pdf
    │   └── Rockchip_User_Guide_J-Link_CN.pdf
    ├── display
    │   ├── Rockchip_Android13_SDK_Developer_Guide_Multi_Screen_Display_CN.pdf
    │   ├── Rockchip_BT656_TX_AND_BT1120_TX_Developer_Guide_CN.pdf
    │   ├── Rockchip_Developer_Guide_Android_Car_MultiDisplay_CN.pdf
    │   ├── Rockchip_Developer_Guide_Baseparameter_Format_Define_And_Use_CN.pdf
    │   ├── Rockchip_Developer_Guide_BT656_TX_AND_BT1120_TX_CN.pdf
    │   ├── Rockchip_Developer_Guide_DisplayPort_CN.pdf
    │   ├── Rockchip_Developer_Guide_DRM_Direct_Show_CN.pdf
    │   ├── Rockchip_Developer_Guide_DRM_Display_Driver_CN.pdf
    │   ├── Rockchip_Developer_Guide_DRM_Panel_Porting_CN.pdf
    │   ├──
Rockchip_Developer_Guide_Dual_Display_Rotation_Direction_Debugging_CN.pdf
    │   ├── Rockchip_Developer_Guide_eDP_CN.pdf
    │   ├── Rockchip_Developer_Guide_HDMI_Based_on_DRM_Framework_CN&EN.pdf
    │   ├── Rockchip_Developer_Guide_HDMI-CEC_CN.pdf
    │   ├── Rockchip_Developer_Guide_HDMI_CN.pdf
    │   ├── Rockchip_Developer_Guide_LVDS_CN.pdf
    │   ├── Rockchip_Developer_Guide_MIPI_DSI2_CN.pdf
    │   ├──
Rockchip_Developer_Guide_MIPI_DSI_Compatible_With_Different_Panels_CN.pdf
    │   ├── Rockchip_Developer_Guide_RGB_MCU_CN.pdf
    │   ├── Rockchip_Developer_Guide_RK628_For_All_Porting_CN.pdf
    │   ├── Rockchip_Developer_Guide_RK628_For_All_Porting_EN.pdf
    │   ├── Rockchip_Developer_Guide_RK628_MCU_CN.pdf
    │   ├── Rockchip_Developer_Guide_RkDisplayOutputManager_CN.pdf
```

```
│   ├── Rockchip_Developer_Guide_RkDisplayOutputManager_EN.pdf
│   ├── Rockchip_DisplayPort_SI_Test_Guide_CN.pdf
│   ├── Rockchip_DRM_RK628_Porting_Guide_CN.pdf
│   ├── Rockchip FAQ DRM Hardware Composer V1.00-20181213.pdf
│   ├── Rockchip_Introduction_DisplayAdjust_APK_CN.pdf
│   ├── Rockchip_RK3399_Developer_Guide_DisplayPort_CN.pdf
│   ├── Rockchip_RK3588_Developer_Guide_DisplayPort_CN.pdf
│   ├── Rockchip_RK3588_Developer_Guide_HDCP_CN.pdf
│   ├── Rockchip_RK3588_Developer_Guide_HDCP_EN.pdf
│   ├── Rockchip_RK3588_Developer_Guide_MIPI_DSI2_CN.pdf
│   ├── Rockchip_RK3588_Developer_Guide_Vsync_Adjust_CN.pdf
│   ├── Rockchip_RK3588_User_Guide_DP_CN.pdf
│   └── Rockchip_RK3588_User_Guide_eDP_CN.pdf
├── DVFS
│   ├── Rockchip_Developer_Guide_CPUFreq_CN.pdf
│   ├── Rockchip_Developer_Guide_CPUFreq_EN.pdf
│   └── Rockchip_Developer_Guide_Devfreq_CN.pdf
├── Ebook
│   ├── Rockchip_RK3566_Introduction_EBOOK_Display_Mode_CN.pdf
│   └── Rockchip_RK3566_Introduction_EBOOK_Sleep_Mode_CN.pdf
├── GMAC
│   ├── Rockchip_Developer_Guide_GMAC_PTP1588_CN.pdf
│   ├── Rockchip_Developer_Guide_Linux_GMAC_CN.pdf
│   ├── Rockchip_Developer_Guide_Linux_GMAC_DPDK_CN.pdf
│   ├── Rockchip_Developer_Guide_Linux_GMAC_Mode_Configuration_CN.pdf
│   ├── Rockchip_Developer_Guide_Linux_GMAC_RGMII_Delayline_CN.pdf
│   ├── Rockchip_Developer_Guide_Linux_GMAC_RGMII_Delayline_EN.pdf
│   └── Rockchip_Developer_Guide_Linux_MAC_TO_MAC_CN.pdf
├── gpio
│   └── Rockchip_Developer_Guide_GPIO_CN.pdf
├── hdmi-in
│   ├── apk
│   │   ├── HdmiInDemo_based_on_CameraHal1_2020.06.11_v1.2.tar.gz
│   │   └── rkCamera2_based_on_CameraHal3_V1.3.tar.gz
│   ├── Rockchip_Developer_Guide_Android12+_HDMI_IN_Bridge_CN.pdf
│   ├── Rockchip_Developer_Guide_HDMI_CEC_CN.pdf
│   ├── Rockchip_Developer_Guide_HDMI_IN_Based_On_CameraHal3_CN.pdf
│   ├── Rockchip_Developer_Guide_HDMI_RX_CN.pdf
│   └── Rockchip_RK3588_HDMI_To_CSI_Dual_Mipi_Developer_Guide.pdf
├── I2C
│   ├── Rockchip_Developer_Guide_I2C_CN.pdf
│   └── Rockchip_Developer_Guide_I2C_EN.pdf
├── IO-Domain
│   ├── Rockchip_Developer_Guide_Linux_IO_DOMAIN_CN.pdf
│   ├── Rockchip_PX30_Introduction_IO_Power_Domains_Configuration.pdf
│   ├── Rockchip_RK3288_Introduction_IO_Power_Domains_Configuration.pdf
│   ├── Rockchip_RK3326_Introduction_IO_Power_Domains_Configuration.pdf
│   ├── Rockchip_RK3399_Introduction_IO_Power_Domains_Configuration.pdf
│   ├── Rockchip_RK3399Pro_Introduction_IO_Power_Domains_Configuration.pdf
│   └── Rockchip_RK356X_Introduction_IO_Power_Domains_Configuration.pdf
├── IOMMU
│   ├── Rockchip_Developer_Guide_Linux_IOMMU_CN.pdf
│   └── Rockchip_Developer_Guide_Linux_IOMMU_EN.pdf
├── MCU
│   ├── Rockchip_RK3399_Developer_Guide_MCU_CN.pdf
│   └── Rockchip_RK3399_Developer_Guide_MCU_EN.pdf
├── Memory
│   ├── Rockchip_Developer_Guide_Linux_CMA_CN.pdf
```

```
    │   ├── Rockchip_Developer_Guide_Linux_DMABUF_CN.pdf
    │   ├── Rockchip_Developer_Guide_Linux_Meminfo_CN.pdf
    │   └── Rockchip_Trouble_Shooting_Linux_Memory_CN.pdf
    ├── MMC
    │   └── Rockchip_Developer_Guide_SDMMC_SDIO_eMMC_CN.pdf
    ├── MPP
    │   ├── Rockchip_Developer_Guide_MPP_CN.pdf
    │   └── Rockchip_Developer_Guide_MPP_EN.pdf
    ├── NVM
    │   ├── Rockchip_Application_Notes_Storage_CN.pdf
    │   ├── Rockchip_Application_Notes_Storage_EN.pdf
    │   ├── Rockchip_Developer_FAQ_Storage_CN.pdf
    │   ├── Rockchip_Developer_Guide_Dual_Storage_CN.pdf
    │   ├── Rockchip_Developer_Guide_OTP_CN.pdf
    │   ├── Rockchip_Developer_Guide_OTP_EN.pdf
    │   ├── Rockchip_Developer_Guide_SATA_CN.pdf
    │   ├── Rockchip_Developer_Guide_UFS_CN.pdf
    │   ├── Rockchip_Introduction_Partition_CN.pdf
    │   ├── Rockchip_Introduction_Partition_EN.pdf
    │   ├── Rockchip_RK3568_Reference_SATA_Performance_CN.pdf
    │   └── Rockchip_Trouble_Shooting_Firmware_Upgrade_Issue_CN.pdf
    ├── PCie
    │   ├── Rockchip_Developer_Guide_PCIe_CN.pdf
    │   ├── Rockchip_Developer_Guide_PCIE_EP_Standard_Card_CN.pdf
    │   ├── Rockchip_Developer_Guide_PCIe_Performance_CN.pdf
    │   ├── Rockchip_Developer_Guide_RT-Thread_PCIe_CN.pdf
    │   ├── Rockchip_PCIe_Virtualization_Developer_Guide_CN.pdf
    │   └── Rockchip_RK3399_Developer_Guide_PCIe_CN.pdf
    ├── perf
    │   ├── Rockchip_Developer_FAQ_FileSystem_CN.pdf
    │   ├──
Rockchip_Developer_Guide_Linux_RealTime_Performance_Test_Report_CN.pdf
    │   ├──
Rockchip_Developer_Guide_Linux_RealTime_Performance_Test_Report_EN.pdf
    │   ├── Rockchip_Optimize_Tutorial_Linux_IO_CN.pdf
    │   ├── Rockchip_Quick_Start_Linux_Perf_CN.pdf
    │   ├── Rockchip_Quick_Start_Linux_Performance_Analyse_CN.pdf
    │   └── Rockchip_Quick_Start_Linux_Systrace_CN.pdf
    ├── PIN-Ctrl
    │   ├── Rockchip_Developer_Guide_Linux_Pinctrl_CN.pdf
    │   └── Rockchip_Developer_Guide_Linux_Pinctrl_EN.pdf
    ├── PMIC
    │   ├── Rockchip_Developer_Guide_FreeRTOS_PMIC_CHARGER_POWERKEY_CN.pdf
    │   ├── Rockchip_Developer_Guide_Power_Discrete_DCDC_EN.pdf
    │   ├── Rockchip_RK805_Developer_Guide_CN.pdf
    │   ├── Rockchip_RK806_Developer_Guide_CN.pdf
    │   ├── Rockchip_RK808_Developer_Guide_CN.pdf
    │   ├── Rockchip_RK809_Developer_Guide_CN.pdf
    │   ├── Rockchip_RK816_Developer_Guide_CN.pdf
    │   ├── Rockchip_RK817_Developer_Guide_CN.pdf
    │   ├── Rockchip_RK818_Developer_Guide_CN.pdf
    │   ├── Rockchip_RK818_RK816_Developer_Guide_Fuel_Gauge_CN.pdf
    │   └── Rockchip_RK818_RK816_Introduction_Fuel_Gauge_Log_CN.pdf
    ├── power
    │   ├── Rockchip_Developer_Guide_Power_Analysis_CN.pdf
    │   ├── Rockchip_Developer_Guide_Power_Analysis_EN.pdf
    │   └── Rockchip_Developer_Guide_Sleep_and_Resume_CN.pdf
    ├── PWM
```

```
    │   ├── Rockchip_Developer_Guide_Linux_PWM_CN.pdf
    │   └── Rockchip_Developer_Guide_Linux_PWM_EN.pdf
    ├── RGA
    │   ├── Rockchip_Developer_Guide_RGA_CN.pdf
    │   ├── Rockchip_Developer_Guide_RGA_EN.pdf
    │   ├── Rockchip_FAQ_RGA_CN.pdf
    │   └── Rockchip_FAQ_RGA_EN.pdf
    ├── RK628
    │   ├── Rockchip_Developer_Guide_RK628F_H_HDMI_IN_CN.pdf
    │   ├── Rockchip_Developer_Guide_RK628_For_All_Porting_CN.pdf
    │   └── Rockchip_RK628D_Application_Notes_CN.pdf
    ├── RKTools manuals
    │   ├── RKIQTool_User_Manual_v1.5-CH.pdf
    │   ├── RKIQTool_User_Manual_v1.5-EN.pdf
    │   ├── RK_Platform_apache_tomcat_ota_Server_Setup_Introduction.rar
    │   ├── Rockchip_Box_Factory_Test_Tool_V2.0.rar
    │   ├──
Rockchip_Introduction_MP_Tool_Upgrading_and_Related_Issues_Debugging_CN.pdf
    │   ├── Rockchip_Introduction_REPO_Mirror_Server_Build_and_Management_CN.pdf
    │   ├── Rockchip_Introduction_WNpctool_Write_Tool_CN.pdf
    │   ├── Rockchip_User_Guide_Box_Factory_Test_Tool_CN.pdf
    │   ├── Rockchip_User_Guide_Keybox_Burning_EN.pdf
    │   ├── Rockchip_User_Guide_MP_Flashing_v1.2_CN.pdf
    │   ├── Rockchip_User_Guide_Production_For_Firmware_Download_CN.pdf
    │   ├── Rockchip_User_Guide_RKDevInfoWriteTool_CN.pdf
    │   ├── Rockchip_User_Guide_RKDevInfoWriteTool_EN.pdf
    │   ├── Rockchip_User_Guide_RK_Platform_MP_Upgrading_CN.pdf
    │   ├── Rockchip_User_Manual_RKIQTool_CN.pdf
    │   ├── Rockchip_User_Manual_RKIQTool_EN.pdf
    │   └── Rockchip_User_Manual_Upgrade_DLL_CN.pdf
    ├── SARADC
    │   ├── Rockchip_Developer_Guide_Linux_SARADC_CN.pdf
    │   └── Rockchip_Developer_Guide_Linux_SARADC_EN.pdf
    ├── security
    │   ├── patch
    │   │   └── u-boot
    │   │       └── 0001-avb-add-embedded-key.patch
    │   ├── RK3399_Efuse_Operation_Instructions_V1.00_EN.pdf
    │   ├── Rockchip_Application_Notes_VendorStorage_EN.pdf
    │   ├── Rockchip_Developer_Guide_Android_SecurityBoot_AVB_CN.pdf
    │   ├── Rockchip_Developer_Guide_Android_SecurityBoot_AVB_EN.pdf
    │   ├── Rockchip_Developer_Guide_Anti_Copy_Board_CN.pdf
    │   ├── Rockchip_Developer_Guide_Crypto_HWRNG_CN.pdf
    │   ├── Rockchip_Developer_Guide_Crypto_HWRNG_EN.pdf
    │   ├── Rockchip_Developer_Guide_OTP_CN.pdf
    │   ├── Rockchip_Developer_Guide_OTP_EN.pdf
    │   ├── Rockchip_Developer_Guide_Secure_Boot_Application_Note_EN.pdf
    │   ├── Rockchip_Developer_Guide_Secure_Boot_for_UBoot_Next_Dev_CN.pdf
    │   ├── Rockchip_Developer_Guide_Secure_Boot_for_UBoot_Next_Dev_EN.pdf
    │   ├── Rockchip_Developer_Guide_TEE_SDK_CN.pdf
    │   ├── Rockchip_Developer_Guide_TEE_SDK_EN.pdf
    │   ├── Rockchip_Introduction_Android_Root_CN.pdf
    │   └── Rockchip_RK3399_User_Guide_SecurityBoot_And_AVB_CN.pdf
    ├── Sensors
    │   └── Rockchip_Developer_Guide_Sensors_CN.pdf
    ├── SPI
    │   ├── Rockchip_Developer_Guide_Linux_SPI_CN.pdf
    │   └── Rockchip_Developer_Guide_Linux_SPI_EN.pdf
```

```
    ├── Thermal
    │   ├── Rockchip_Developer_Guide_Thermal_CN.pdf
    │   └── Rockchip_Developer_Guide_Thermal_EN.pdf
    ├── TRUST
    │   ├── Rockchip_Developer_Guide_Trust_CN.pdf
    │   ├── Rockchip_Developer_Guide_Trust_EN.pdf
    │   ├── Rockchip_RK3308_Developer_Guide_System_Suspend_CN.pdf
    │   ├── Rockchip_RK3308_Developer_Guide_System_Suspend_EN.pdf
    │   ├── Rockchip_RK3399_Developer_Guide_System_Suspend_CN.pdf
    │   ├── Rockchip_RK3506_Developer_Guide_System_Suspend_CN.pdf
    │   ├── Rockchip_RK356X_Developer_Guide_System_Suspend_CN.pdf
    │   ├── Rockchip_RK3576_Developer_Guide_System_Suspend_CN.pdf
    │   └── Rockchip_RK3588_Developer_Guide_System_Suspend_CN.pdf
    ├── Tutorial
    │   ├── Rockchip_Developer_FAQ_Browser_CN.pdf
    │   ├── Rockchip_Developer_FAQ_FileSystem_CN.pdf
    │   └── Rockchip_Trouble_Shooting_Firmware_Upgrade_Issue_CN.pdf
    ├── UART
    │   ├── Rockchip_Developer_Guide_RT_Thread_UART.pdf
    │   ├── Rockchip_Developer_Guide_UART_CN.pdf
    │   ├── Rockchip_Developer_Guide_UART_EN.pdf
    │   └── Rockchip_Developer_Guide_UART_FAQ_CN.pdf
    ├── u-boot
    │   ├── Rockchip_Developer_Guide_Linux_AB_System_CN.pdf
    │   ├── Rockchip_Developer_Guide_UBoot_MMC_Device_Analysis_CN.pdf
    │   ├── Rockchip_Developer_Guide_UBoot_Nextdev_CN.pdf
    │   └── Rockchip_Developer_Guide_UBoot_Nextdev_EN.pdf
    ├── UFS
    │   └── Rockchip_Developer_Guide_UFS_CN.pdf
    ├── usb
    │   ├──
Rockchip_Developer_Guide_Linux_USB_Initialization_Log_Analysis_CN.pdf
    │   ├── Rockchip_Developer_Guide_Linux_USB_Performance_Analysis_CN.pdf
    │   ├── Rockchip_Developer_Guide_USB2_Compliance_Test_CN.pdf
    │   ├── Rockchip_Developer_Guide_USB_CN.pdf
    │   ├── Rockchip_Developer_Guide_USB_EN.pdf
    │   ├── Rockchip_Developer_Guide_USB_FFS_Test_Demo_CN.pdf
    │   ├── Rockchip_Developer_Guide_USB_Gadget_UAC_CN.pdf
    │   ├── Rockchip_Developer_Guide_USB_SQ_Test_CN.pdf
    │   ├── Rockchip_Introduction_USB_SQ_Tool_CN.pdf
    │   ├── Rockchip_RK3399_Developer_Guide_USB_CN.pdf
    │   ├── Rockchip_RK356x_Developer_Guide_USB_CN.pdf
    │   ├── Rockchip_RK3576_Developer_Guide_USB_CN.pdf
    │   ├── Rockchip_RK3588_Developer_Guide_USB_CN.pdf
    │   ├── Rockchip_Trouble_Shooting_Linux4.19_USB_Gadget_UVC_CN.pdf
    │   └── Rockchip_Trouble_Shooting_Linux_USB_Host_UVC_CN.pdf
    └── watchdog
        ├── Rockchip_Developer_Guide_Linux_WDT_CN.pdf
        └── Rockchip_Developer_Guide_Linux_WDT_EN.pdf
```

10. 工具使用

10.1 StressTest

设备上使用Stresstest 工具，对待测设备的各项功能进行压力测试，确保整个系统运行的稳定性。SDK通过打开计算器应用，输入`“83991906=”`暗码，可启动StressTest应用，进行各功能压力测试。
Stresstest 测试工具测试的内容主要包括：

10.1.1 模块相关

- Camera 压力测试：包括Camera 打开关闭，Camera 拍照以及Camera 切换。
- Bluetooth 压力测试：包括Bluetooth 打开关闭。
- Wi-Fi 压力测试：包括Wi-Fi 打开关闭，（ping 测试以及iperf 测试待加入）。

10.1.2 非模块相关

- 飞行模式开关测试
- 休眠唤醒拷机测试
- 视频拷机测试
- 重启拷机测试
- 恢复出厂设置拷机测试
- Arm 变频测试
- Gpu 变频测试
- DDR 变频测试

10.2 PCBA测试工具

PCBA 测试工具用于帮助在量产的过程中快速地甄别产品功能的好坏，提高生产效率。目前包括屏幕（LCD）、无线（Wi-Fi）、蓝牙（bluetooth）、DDR/EMMC 存储、SD 卡（sdcard）、USB HOST、按键（KEY），喇叭耳机（Codec）测试项目。
这些测试项目包括自动测试项和手动测试项，无线网络、DDR/EMMC、以太网为自动测试项，按键、SD卡、USB HOST、Codec，手动测试项目。
具体PCBA功能配置及使用说明，请参考：

```
RKDocs/android/Rockchip_Developer_Guide_PCBA_Test_Tool_CN&EN.pdf_V1.1_20171222.pdf
```

10.3 DeviceTest

DeviceTest 用于工厂整机测试，主要测试装成整机以后外围器件是否正常。SDK 通过打开计算器，输入暗码“000.=”进入 DeviceTest，如下所示：



在产线可以根据这个界面进行对应外设的测试，测试时点击“TEST CHECKED”对所测项目逐项进行测试，测试如果成功点击 pass，失败点击 failed，最终结果会显示在界面上，如下图所示，红色为 failed 项，其余为通过项，工厂可根据测试结果进行相应的维修。另外，如果客户需要对该工具进行定制，请联系 FAE 窗口申请对应的源码。

10.4 USB驱动

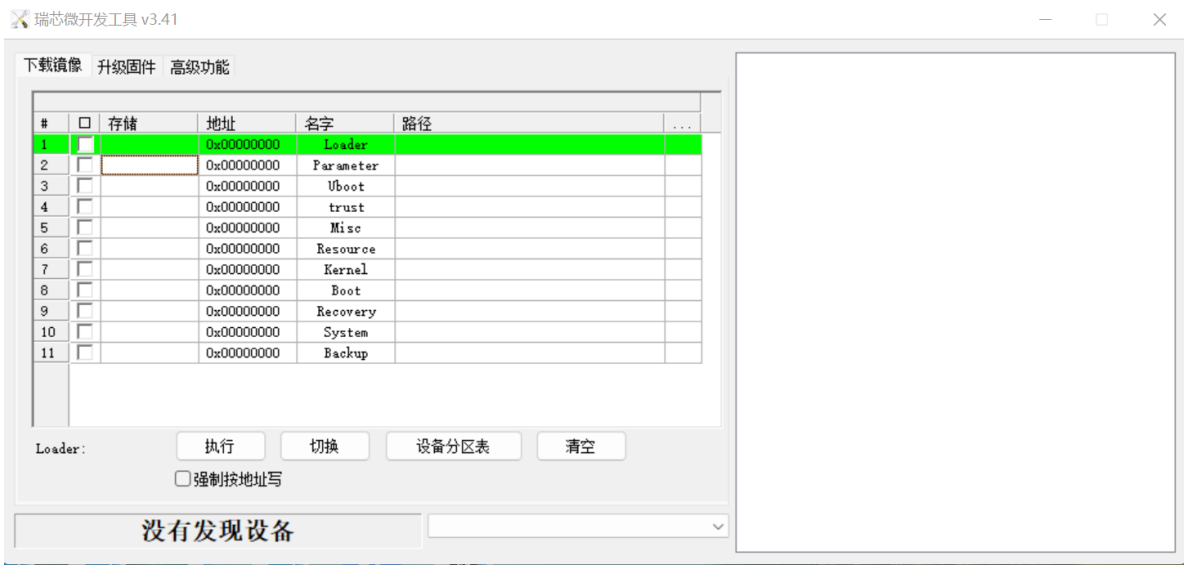
Rockchip USB驱动安装包，包括ADB、固件烧写驱动

```
RKTools\windows\DriverAssitant_v5.14.zip
```

10.5 开发烧写工具

10.5.1 Windows版本

RKTools/windows/AndroidTool/AndroidTool_Release_v3.41.zip，工具版本会时刻更新，请及时同步更新



10.5.2 Linux版本

RKTools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool_v2.55.zip

```
Linux_Upgrade_Tool_v2.26$ sudo ./upgrade_tool -h
Program Data in ~/.config/upgrade_tool

----------------------Tool Usage ----------------------
Help:             H
Quit:             Q
Version:          V
Clear Screen:     CS
------------------Upgrade Command ------------------
ChooseDevice:           CD
ListDevice:                 LD
SwitchDevice:           SD
UpgradeFirmware:        UF <Firmware> [-noreset]
UpgradeLoader:          UL <Loader> [-noreset]
DownloadImage:          DI <-p|-b|-k|-s|-r|-m|-u|-t|-re image>
DownloadBoot:           DB <Loader>
EraseFlash:             EF <Loader|firmware> [DirectLBA]
PartitionList:          PL
WriteSN:                SN <serial number>
ReadSN:             RSN
----------------Professional Command -----------------
TestDevice:             TD
ResetDevice:            RD [subcode]
ResetPipe:              RP [pipe]
ReadCapability:         RCB
ReadFlashID:            RID
ReadFlashInfo:          RFI
ReadChipInfo:           RCI
ReadSector:             RS  <BeginSec> <SectorLen> [-decode] [File]
WriteSector:            WS  <BeginSec> <File>
ReadLBA:                RL  <BeginSec> <SectorLen> [File]
WriteLBA:               WL  <BeginSec> <File>
EraseLBA:               EL  <BeginSec> <EraseCount>
EraseBlock:             EB <CS> <BeginBlock> <BlokcLen> [--Force]
```

```
------------------------------------------------------
```

10.6 SD升级启动制作工具

用于制作SD卡升级、SD卡启动、SD卡PCBA测试

```
RKTools\windows\SDDiskTool_v1.74.zip
```

10.7 写号工具

RKTools\windows\RKDevInfoWriteTool-1.3.0.7z

解压RKDevInfoWriteTool-1.3.0.7z后安装

以管理员权限打开软件

工具说明请参考：

```
RKDocs/common/RKTools manuals\Rockchip_User_Guide_RKDevInfoWriteTool_CN.pdf
```

10.8 DDR焊接测试工具

用于测试DDR的硬件连接，排查虚焊等硬件问题

```
RKTools\windows\Rockchip_Platform_DDR_Test_Tool_V1.38_Release_Annoucement_CN.7z
RKTools\windows\Rockchip_Platform_DDR_Test_Tool_V1.38_Release_Annoucement_EN.7z
```

10.9 efuse烧写工具

用于efuse的烧写，适用于RK3288W/RK3368/RK3399平台

```
RKTools\windows\efuse_v1.37.rar
```

10.10 efuse/otp签名工具

用于固件的efuse/otp签名

```
RKTools\windows\SecureBootTool_v1.94.zip
```

10.11 工厂生产固件烧写工具

用于工厂批量烧写固件

```
RKTools\windows\FactoryTool-1.72.9.7z
```

10.12 userdata分区数据预置工具

用于制作userdata分区预置数据包的工具

```
RKTools\windows\OemTool_v1.3.rar
```

10.13 Camera IQ Tool

用于调试ISP图像效果

```
external/camera_engine_rkaiq/rkisp2x_tuner
```

11. 系统调试

11.1 ADB工具

11.2 概述

ADB（Android Debug Bridge）是 Android SDK里的一个工具，用这个工具可以操作管理 Android 模拟器或真实的 Android 设备。主要功能有：

- 运行设备的 shell（命令行）
- 管理模拟器或设备的端口映射
- 计算机和设备之间上传/下载文件
- 将本地 apk软件安装至模拟器或 Android 设备

ADB 是一个“客户端－服务器端”程序，其中客户端主要是指PC，服务器端是Android 设备的实体机器或者虚拟机。根据PC连接Android设备的方式不同，ADB 可以分为两类：

- 网络 ADB：主机通过有线/无线网络（同一局域网）连接到STB设备
- USB ADB：主机通过 USB 线连接到STB设备

11.2.1 USB adb使用说明

USB adb 使用有以下限制：

- 只支持 USB OTG 口
- 不支持多个客户端同时使用（如 cmd 窗口，eclipse等）
- 只支持主机连接一个设备，不支持连接多个设备

连接步骤如下：

1. Android设备已经运行 Android 系统，设置->开发者选项->已连接到计算机 打开，usb调试开关打开。
2. PC主机只通过 USB 线连接到机器 USB otg 口，然后电脑通过如下命令与Android设备相连。

```
adb shell
```

3. 测试是否连接成功，运行”adb devices”命令，如果显示机器的序列号，表示连接成功。

11.3 ADB常用命令详解

1. 查看设备情况

查看连接到计算机的 Android 设备或者模拟器：

```
adb devices
```

返回的结果为连接至开发机的 Android 设备的序列号或是IP和端口号（Port）、状态。

2. 安装apk

将指定的 apk 文件安装到设备上：

```
adb install <apk文件路径>
```

示例如下：

```
adb install “F:\WishTV\WishTV.apk”
```

重新安装应用：

```
adb install -r "F:\WishTV\WishTV.apk"
```

3. 卸载apk

完全卸载：

```
adb uninstall <package>
```

示例如下：

```
adb uninstall com.wishtv
```

4. 使用 rm移除 apk 文件：

```
adb shell rm <filepath>
```

示例如下：

```
adb shell rm "system/app/WishTV.apk"
```

示例说明：移除“system/app”目录下的“WishTV.apk”文件。

5. 进入设备和模拟器的shell

进入设备或模拟器的 shell 环境：

```
adb shell
```

6. 从电脑上传文件到设备

用 push 命令可以把本机电脑上的任意文件或者文件夹上传到设备。本地路径一般指本机电脑；远程路径一般指 adb 连接的单板设备。
adb push <本地路径> <远程路径>
示例如下：

```
adb push "F:\WishTV\WishTV.apk" "system/app"
```

示例说明：将本地“WishTV.apk”文件上传到 Android 系统的“system/app”目录下。

7. 从设备下载文件到电脑

pull 命令可以把设备上的文件或者文件夹下载到本机电脑中。

```
adb pull <远程路径> <本地路径>
```

示例如下：

```
adb pull system/app/Contacts.apk F:\
```

示例说明：将 Android 系统“system/app”目录下的文件或文件夹下载到本地“F:\”目录下。

8. 查看 bug报告

需要查看系统生成的所有错误消息报告，可以运行 adb bugreport指令来实现，该指令会将 Android 系统的dumpsys、dumpstate 与 logcat 信息都显示出来。

9. 查看设备的系统信息

在 adb shell下查看设备系统信息的具体命令。

```
adb shell getprop
```

11.4 Logcat工具

Android 日志系统提供了记录和查看系统调试信息的功能。日志都是从各种软件和一些系统的缓冲区中记录下来的，缓冲区可以通过 Logcat 来查看和使用。Logcat 是调试程序用的最多的功能。该功能主要是通过打印日志来显示程序的运行情况。由于要打印的日志量非常大，需要对其进行过滤等操作。

11.4.1 Logcat命令使用

用 logcat 命令来查看系统日志缓冲区的内容：
基本格式：

```
[adb] logcat [<option>] [<filter-spec>]
```

示例如下：

```
adb shell
logcat
```

11.5 常用的日志过滤方式

控制日志输出的几种方式：

- 控制日志输出优先级

示例如下：

```
adb shell
logcat *:W
```

示例说明：显示优先级为 warning 或更高的日志信息。

- 控制日志标签和输出优先级

示例如下：

```
adb shell
logcat ActivityManager:I MyApp:D *:S
```

示例说明：支持所有的日志信息，除了那些标签为“ActivityManager”和优先级为“Info”以上的、标签为“MyApp”和优先级为“Debug”以上的。

- 只输出特定标签的日志

示例如下：

```
adb shell
logcat WishTV:* *:S
```

或者

```
adb shell
logcat -s WishTV
```

示例说明：只输出标签为 WishTV的日志。

- 只输出指定优先级和标签的日志

示例如下：

```
adb shell
logcat WishTV:I *:S
```

示例说明：只输出优先级为 I，标签为 WishTV的日志。

11.6 Procrank工具

Procrank 是 Android 自带的一款调试工具，运行在设备侧的 shell 环境下，用来输出进程的内存快照，便于有效的观察进程的内存占用情况。
包括如下内存信息：

- VSS：Virtual Set Size 虚拟耗用内存大小（包含共享库占用的内存）
- RSS：Resident Set Size 实际使用物理内存大小（包含共享库占用的内存）
- PSS：Proportional Set Size 实际使用的物理内存大小（比例分配共享库占用的内存）
- USS：Unique Set Size 进程独自占用的物理内存大小（不包含共享库占用的内存）

注意：

- USS 大小代表只属于本进程正在使用的内存大小，进程被杀死后会被完整回收；
- VSS/RSS 包含了共享库使用的内存，对查看单一进程内存状态没有参考价值；
- PSS 是按照比例将共享内存分割后，某单一进程对共享内存区的占用情况。

11.6.1 使用procrank

执行procrank前需要先让终端获取到root权限
su
命令格式：

```
procrank [ -W ] [ -v | -r | -p | -u | -h ]
```

常用指令说明：
-v：按照 VSS 排序
-r：按照 RSS 排序
-p：按照 PSS 排序
-u：按照 USS 排序
-R：转换为递增[递减]方式排序
-w：只显示 working set 的统计计数
-W：重置 working set 的统计计数
-h：帮助
示例：
输出内存快照：

```
procrank
```

按照 VSS 降序排列输出内存快照：

```
procrank -v
```

默认procrank输出是通过PSS排序。

11.6.2 检索指定内容信息

查看指定进程的内存占用状态，命令格式如下：

```
procrank | grep [cmdline | PID]
```

其中 cmdline 表示需要查找的应用程序名，PID 表示需要查找的应用进程。
输出 systemUI 进程的内存占用状态：

```
procrank | grep “com.android.systemui”
```

或者：

```
procrank  | grep 3396
```

11.6.3 跟踪进程内存状态

通过跟踪内存的占用状态，进而分析进程中是否存在内存泄露场景。使用编写脚本的方式，连续输出进程的内存快照，通过对比 USS 段，可以了解到此进程是否有内存泄露。
示例：输出进程名为 com.android.systemui 的应用内存占用状态，查看是否有泄露：

1. 编写脚本 test.sh

```
#!/bin/bash
while true;do
adb shell procrank | grep “com.android.systemui”
sleep 1
done
```

2. 通过 adb 工具连接到设备后，运行此脚本：./test.sh

11.7 Dumpsys工具

Dumpsys 工具是 Android系统中自带的一款调试工具，运行在设备侧的 shell 环境下，提供系统中正在运行的服务状态信息功能。正在运行的服务是指 Android binder机制中的服务端进程。
dumpsys 输出打印的条件：

1. 只能打印已经加载到 ServiceManager中的服务；
2. 如果服务端代码中的 dump 函数没有被实现，则没有信息输出。

11.7.1 使用Dumpsys

- 查看Dumpsys帮助

作用：输出dumpsys帮助信息。

```
dumpsys  -help
```

- 查看Dumpsys包含服务列表

作用：输出dumpsys所有可打印服务信息，开发者可以关注需要调试服务的名称。

```
dumpsys  -l
```

- 输出指定服务的信息

作用：输出指定的服务的 dump 信息。
格式：dumpsys [servicename]
示例：输出服务 SurfaceFlinger的信息，可执行命令：

```
dumpsys SurfaceFlinger
```

- 输出指定服务和应有进程的信息

作用：输出指定服务指定应用进程信息。
格式：dumpsys [servicename] [应用名]
示例：输出服务名为 meminfo，进程名为 com.android.systemui 的内存信息，执行命令：

```
dumpsys meminfo com.android.systemui
```

注意：服务名称是大小写敏感的，并且必须输入完整服务名称。

11.8 Last log 开启

- 在dts文件里面添加下面两个节点

```
ramoops_mem: ramoops_mem {
    reg = <0x0 0x110000 0x0 0xf0000>;
    reg-names = "ramoops_mem";
};

ramoops {
    compatible = "ramoops";
    record-size = <0x0 0x20000>;
    console-size = <0x0 0x80000>;
    ftrace-size = <0x0 0x00000>;
    pmsg-size = <0x0 0x50000>;
    memory-region = <&ramoops_mem>;
```

```
};
```

```
- 在机器中查看last log

130|root@rk3399:/sys/fs/pstore # ls
dmesg-ramoops-0    上次内核panic后保存的log。
pmsg-ramoops-0     上次用户空间的log，android的log。
ftrace-ramoops-0     打印某个时间段内的function trace。
console-ramoops-0  last_log 上次启动的kernel log，但只保存了优先级比默认log level 高的
log。
```

- 使用方法：

```
cat dmesg-ramoops-0
cat console-ramoops-0
logcat -L （pmsg-ramoops-0） 通过logcat 取出来并解析pull out by logcat and parse
cat ftrace-ramoops-0
```

11.9 FIQ模式

当设备死机或者卡住的时候可以在串口输入fiq命令查看系统的状态，具体命令如下：

```
127|console:/ $ fiq
debug>  help
FIQ Debugger commands:
 pc             PC status
 regs           Register dump
 allregs        Extended Register dump
 bt             Stack trace
 reboot [<c>]   Reboot with command <c>
 reset [<c>]    Hard reset with command <c>
 irqs           Interrupt status
 kmsg           Kernel log
 version        Kernel version
 sleep          Allow sleep while in FIQ
 nosleep        Disable sleep while in FIQ
 console        Switch terminal to console
 cpu            Current CPU
 cpu <number>   Switch to CPU<number>
 ps             Process list
 sysrq          sysrq options
 sysrq <param>  Execute sysrq with <param>
```

12. 常见问题

12.1 当前kernel和u-boot版本？

```
Android16.0 对应的kernel大版本为：6.12，u-boot的分支为v5分支
```

U-Boot 版本确认（通过源码）：

U-Boot 源码位于 SDK 的 `u-boot` 目录下。

- 分支名称： `v5`（manifest 中指定 `revision="v5"`）
- **U-Boot 版本：** `2025.04`（基于 upstream v2025.04）
- 最近 **tag：** `linux-6.12-stan-rkb0-45-gc7d5af87a6a`（包含 Rockchip 平台适配补丁）
- 远程仓库： `ssh://gerrit-server:29418/android/rk/u-boot`

查看本地 u-boot 版本信息：

```
cd u-boot
git branch -a           # 查看分支，当前分支应为 v5
git describe --tags     # 查看当前对应 tag
grep -E 'VERSION|PATCHLEVEL|SUBLEVEL' Makefile  # 查看 U-Boot 版本号
```

RK3572 的 U-Boot defconfig：

```
u-boot/configs/rk3572_defconfig
```

该 defconfig 默认 Device Tree 为 `rockchip/rk3572-evb`，对应 U-Boot 源码中的 `dts/upstream/src/arm64/rockchip/rk3572-evb.dts`。

12.2 如何获取当前SDK对应的RK release版本

Rockchip Android16.0 SDK包括AOSP原始代码和RK修改的代码两部分，其中RK修改的仓库包含在`.repo/manifests/include`目录下面的xml中，AOSP默认的仓库在`.repo/manifests/default.xml`。
版本确认：

- RK修改部分

```
vim .repo/manifests/include/rk_checkout_from_aosp.xml
<project groups="pdk" name="platform/build" path="build/make" remote="rk"
revision="refs/tags/android-16.0-mid-rkr5">
```

说明RK的版本是android-16.0-mid-rkr5

- AOSP部分

```
vim .repo/manifests/default.xml
<default revision="refs/tags/android-16.0.0_r1"...>
```

说明AOSP的版本是android-16.0.0_r1
当需要提供版本信息的时候提供以上两个版本信息即可。
单个仓库可以直接通过如下命令获取tag信息

```
kernel-6.12$ git tag
android-16.0-mid-rkr5
```

RK的版本是以android-16.0-mid-rkrxx的格式递增的，所以当前的最新tag是android-16.0-mid-rkr5

12.3 如何确认本地SDK已经完整更新RK发布的SDK状态

RK发布SDK版本时会在.repo/manifests/commit/目录下对应提交该版本的commit信息，客户可以通过对比这个commit信息来确认是否有完整更新SDK，具体操作如下：

- 按“如何获取当前SDK对应的RK release版本”的说明先确认SDK的RK版本，下面以RK版本是RKR1为例进行说明；
- 用如下命令保存本地的commit信息

```
.repo/repo/repo manifest -r -o release_manifest_rkr1_local.xml
```

- 通过比较.repo/manifests/commit/commit_release_rkr1.xml和release_manifest_rkr1_local.xml，即可确认SDK代码是否更新完整，

其中.repo/manifests/commit/commit_release_rkr1.xml为RK版本RKR1发布的commit信息。

12.4 uboot和kernel阶段logo图片替换

uboot和kernel阶段的logo分别为开机显示的第一张和第二张logo图片，可以根据产品需求进行修改替换。
uboot logo源文件：`kernel-6.12/logo.bmp`
kernel logo源文件：`kernel-6.12/logo_kernel.bmp`
如果需要更换某一张，只需用同名的bmp替换掉，重新编译内核即可，编译后的文件在boot.img中。
说明：Logo图片大小目前只支持到8M以内大小的bmp格式图片，支持8、16、24、32位的bmp。

12.5 如何修改Android系统仅支持64位系统

Android14开始过GMS和EDLA认证的产品要求配置为仅支持64位的系统，不再支持32位的。修改为仅支持64位的系统，可以减少内存占用，具体修改如下(以rk3572_a16为例说明)：
在产品目录的BoardConfig.mk中修改对应的配置，如下

```
diff --git a/rk3572_a16/BoardConfig.mk b/rk3572_a16/BoardConfig.mk
index c06433e..dc9cc50 100644
--- a/rk3572_a16/BoardConfig.mk
+++ b/rk3572_a16/BoardConfig.mk
@@ -18,3 +18,11 @@ PRODUCT_KERNEL_DTS := rk3572-evb1-v10
 CAMERA_SUPPORT_AUTOFOCUS := true
 BOARD_BUILD_GKI := true
 include device/rockchip/rk3572/rk3572_a16/BoardConfig.mk
+
+DEVICE_IS_64BIT_ONLY := true
+
+TARGET_2ND_ARCH :=
+TARGET_2ND_ARCH_VARIANT :=
+TARGET_2ND_CPU_ABI :=
+TARGET_2ND_CPU_ABI2 :=
+TARGET_2ND_CPU_VARIANT :=
```

12.6 关机充电和低电预充

关机充电和低电预充可以在dts中配置，具体如下：

```
charge-animation {
                compatible = "rockchip,uboot-charge";
                rockchip,uboot-charge-on = <1>;
                rockchip,android-charge-on = <0>;
                rockchip,uboot-low-power-voltage = <3400>;
                rockchip,screen-on-voltage = <3500>;
                status = "okay";
        };
```

其中：
rockchip,uboot-charge-on ： uboot关机充电，与android关机充电互斥
rockchip,android-charge-on ： android关机充电，与uboot关机充电互斥
rockchip,uboot-low-power-voltage ： 配置低电预充到开机的电压，可以根据实际需求进行配置
rockchip,screen-on-voltage：配置低电预充到亮屏的电压，可以根据实际需求进行配置

12.7 Uboot阶段充电图片打包和替换

充电图片路径，可以直接替换同名文件，格式要求与原文件一样。

```
u-boot/tools/images/
├── battery_0.bmp
├── battery_1.bmp
├── battery_2.bmp
├── battery_3.bmp
├── battery_4.bmp
├── battery_5.bmp
└── battery_fail.bmp
```

如果打开uboot充电，但是没有显示充电图片，可能是图片没有打包到resource.img中，可以按如下命令打包：

```
cd u-boot
./scripts/pack_resource.sh ../kernel-6.12/resource.img
cp resource.img ../kernel-6.12/resource.img
```

执行以上命令后uboot充电图片会打包到kernel目录的resource.img中，此时需要再将resource.img打包到boot.img中，可以在android根目录执行./mkimage.sh，然后烧写rockdev/下面的boot.img即可。

12.8 HDMI IN配置

hdmi in功能SDK默认是关闭的，如需打开，按以下操作：

```
vim device/rockchip/rk3572/BoardConfig.mk
+BOARD_HDMI_IN_SUPPORT := true
```

12.9 RM310 4G配置

4G功能SDK默认是关闭的，如需打开，按以下操作：

```
vim device/rockchip/common/BoardConfig.mk
#for rk 4g modem
-BOARD_HAS_RK_4G_MODEM ?= false
+BOARD_HAS_RK_4G_MODEM ?= true
```

12.10 Wi-Fi休眠策略配置

Wi-Fi默认休眠策略是休眠一直保持连接，如需休眠断开，按以下操作：

```
---
a/rk3566_rgo/overlay/frameworks/base/packages/SettingsProvider/res/values/defaul
ts.xml
+++
b/rk3566_rgo/overlay/frameworks/base/packages/SettingsProvider/res/values/defaul
ts.xml
@@ -24,5 +24,5 @@
        You can configure persist.wifi.sleep.delay.ms to delay closing wifi.
        The default is 15 minutes, 0 means that the wifi is turned off
        immediately after the screen is off. -->

-    <integer name="def_wifi_sleep_policy">2</integer>

+    <integer name="def_wifi_sleep_policy">0</integer>
 </resources>
```

12.11 Recovery旋转配置

支持Recovery旋转0/90/180/270度，默认不旋转（即旋转0度），旋转配置说明如下：

```
vim device/rockchip/common/BoardConfig.mk
#0:   ROTATION_NONE 旋转0度
#90:  ROTATION_RIGHT 旋转90度
#180: ROTATION_DOWN 旋转180度
#270: ROTATION_LEFT 旋转270度
# For Recovery Rotation
TARGET_RECOVERY_DEFAULT_ROTATION ?= ROTATION_NONE
```

12.12 Android Surface旋转

Android系统显示旋转，可以修改如下配置，配置参数为0/90/180/270

```
# For Surface Flinger Rotation
SF_PRIMARY_DISPLAY_ORIENTATION ?= 0
```

12.13 替换 AOSP 部分源代码的 remote

客户下载RK的release代码时速度较慢，可以将AOSP的remote修改为国内镜像源，国外的客户可以修改为Google的镜像源。这样可以提高下载速度。具体操作方法如下：
执行repo init（或者解压base包）后，修改.repo/manifests/remote.xml，把其中的 AOSP 这个 remote 的 fetch 从

```
<remote name="aosp" fetch="./" review="https://gerrit-server" />
```

改为
国内客户：（国内以清华大学镜像源为例，可以根据需要修改为其他国内镜像源)

```
<remote name="aosp" fetch="https://aosp.tuna.tsinghua.edu.cn" />
```

国外的客户：（Google镜像源）

```
<remote name="aosp" fetch="https://android.googlesource.com" />
```

12.14 Data区读写速率的优化

针对带电池的设备，建议fstab的data分区挂载参数加上‘fsync_mode=nobarrier’，可以较大地提升存储读写速率，提升性能。这个参数对不带电池的设备存在掉电数据损害的风险，所以不建议不带电池的设备加这个参数。修改补丁如下：

```
cd device/rockchip/common

diff --git a/scripts/fstab_tools/fstab.in b/scripts/fstab_tools/fstab.in
index 2ec6c265..c890cc84 100755
--- a/scripts/fstab_tools/fstab.in
+++ b/scripts/fstab_tools/fstab.in
@@ -23,6 +23,6 @@ ${_block_prefix}odm     /odm      ext4 ro,barrier=1
${_flags},first_stage_mount
 # For sdmmc
 /devices/platform/${_sdmmc_device}/mmc_host*        auto  auto    defaults
   voldmanaged=sdcard1:auto
 #  Full disk encryption has less effect on rk3326, so default to enable this.
-/dev/block/by-name/userdata /data f2fs
noatime,nosuid,nodev,discard,reserve_root=32768,resgid=1065
latemount,wait,check,fileencryption=aes-256-xts:aes-256-
cts:v2+inlinecrypt_optimized,keydirectory=/metadata/vold/metadata_encryption,quo
ta,formattable,reservedsize=128M,checkpoint=fs
+/dev/block/by-name/userdata /data f2fs
noatime,nosuid,nodev,discard,reserve_root=32768,resgid=1065,fsync_mode=nobarrier
latemount,wait,check,fileencryption=aes-256-xts:aes-256-
cts:v2+inlinecrypt_optimized,keydirectory=/metadata/vold/metadata_encryption,quo
ta,formattable,reservedsize=128M,checkpoint=fs
 # for ext4
 #/dev/block/by-name/userdata    /data      ext4
discard,noatime,nosuid,nodev,noauto_da_alloc,data=ordered,user_xattr,barrier=1
latemount,wait,formattable,check,fileencryption=software,quota,reservedsize=128M
,checkpoint=block
diff --git a/scripts/fstab_tools/fstab_go.in b/scripts/fstab_tools/fstab_go.in
index 582557f2..05c7653c 100755
```

```
--- a/scripts/fstab_tools/fstab_go.in
+++ b/scripts/fstab_tools/fstab_go.in
@@ -17,6 +17,6 @@ ${_block_prefix}odm     /odm      ext4 ro,barrier=1
${_flags},first_stage_mount
 # For sdmmc
 /devices/platform/${_sdmmc_device}/mmc_host*        auto  auto    defaults
   voldmanaged=sdcard1:auto
 #  Full disk encryption has less effect on rk3326, so default to enable this.
-/dev/block/by-name/userdata /data f2fs
noatime,nosuid,nodev,discard,reserve_root=32768,resgid=1065
latemount,wait,check,fileencryption=aes-256-xts:aes-256-
cts:v2+inlinecrypt_optimized,keydirectory=/metadata/vold/metadata_encryption,quo
ta,formattable,reservedsize=128M,checkpoint=fs
+/dev/block/by-name/userdata /data f2fs
noatime,nosuid,nodev,discard,reserve_root=32768,resgid=1065,fsync_mode=nobarrier
latemount,wait,check,fileencryption=aes-256-xts:aes-256-
cts:v2+inlinecrypt_optimized,keydirectory=/metadata/vold/metadata_encryption,quo
ta,formattable,reservedsize=128M,checkpoint=fs
 # for ext4
 #/dev/block/by-name/userdata    /data      ext4
discard,noatime,nosuid,nodev,noauto_da_alloc,data=ordered,user_xattr,barrier=1
latemount,wait,formattable,check,fileencryption=software,quota,reservedsize=128M
,checkpoint=block
```

12.15 userdata区文件系统换为EXT4

默认data分区的文件系统为f2fs，建议不带电池的产品可以将data区的文件系统改为ext4，可以减小异常掉电后数据丢失的概率。修改方法如下：
以rk3566_r为例说明：

```
device/rockchip/common$ git diff
diff --git a/scripts/fstab_tools/fstab.in b/scripts/fstab_tools/fstab.in
index 6e78b00..a658332 100755
--- a/scripts/fstab_tools/fstab.in
+++ b/scripts/fstab_tools/fstab.in
@@ -20,6 +20,6 @@ ${_block_prefix}system_ext /system_ext  ext4 ro,barrier=1
${_flags},first_stage_
 # For sdmmc
 /devices/platform/${_sdmmc_device}/mmc_host*        auto  auto    defaults
   voldmanaged=sdcard1:auto
 #  Full disk encryption has less effect on rk3326, so default to enable this.
-/dev/block/by-name/userdata /data f2fs
noatime,nosuid,nodev,discard,reserve_root=32768,resgid=1065
latemount,wait,check,fileencryption=aes-256-xts:aes-256-
cts:v2+inlinecrypt_optimized,quota,formattable,reservedsize=128M,checkpoint=fs
+#/dev/block/by-name/userdata /data f2fs
noatime,nosuid,nodev,discard,reserve_root=32768,resgid=1065
latemount,wait,check,fileencryption=aes-256-xts:aes-256-
cts:v2+inlinecrypt_optimized,quota,formattable,reservedsize=128M,checkpoint=fs
 # for ext4
```

```
-#/dev/block/by-name/userdata     /data       ext4
discard,noatime,nosuid,nodev,noauto_da_alloc,data=ordered,user_xattr,barrier=1
latemount,wait,formattable,check,fileencryption=software,quota,reservedsize=128M
,checkpoint=block
+/dev/block/by-name/userdata     /data       ext4
discard,noatime,nosuid,nodev,noauto_da_alloc,data=ordered,user_xattr,barrier=1
latemount,wait,formattable,check,fileencryption=software,quota,reservedsize=128M
,checkpoint=block
```

```
device/rockchip/rk356x$ git diff
diff --git a/rk3566_r/recovery.fstab b/rk3566_r/recovery.fstab
index 7532217..cf789ac 100755
--- a/rk3566_r/recovery.fstab
+++ b/rk3566_r/recovery.fstab
@@ -7,7 +7,7 @@
 /dev/block/by-name/odm                  /odm                ext4
defaults                  defaults
 /dev/block/by-name/cache                /cache              ext4
defaults                  defaults
 /dev/block/by-name/metadata             /metadata           ext4
defaults                  defaults
-/dev/block/by-name/userdata             /data               f2fs
defaults                  defaults
+/dev/block/by-name/userdata             /data               ext4
defaults                  defaults
 /dev/block/by-name/cust                 /cust               ext4
defaults                  defaults
 /dev/block/by-name/custom               /custom             ext4
defaults                  defaults
 /dev/block/by-name/radical_update       /radical_update     ext4
defaults                  defaults
```

12.16 修改开关机动画和开关机铃声

参考文档：

```
RKDocs/android/Rockchip_Introduction_Android_Power_On_Off_Animation_and_Tone_Cus
tomization_CN&EN.pdf
```

12.17 APP设置性能模式

device/rockchip/rk3xxx/下配置文件：package_performance.xml，在其中的节点中加入需要使用性能模式的包名：（使用 aapt dump badging (file_path.apk)获取包名）

```
<app package="包名" mode="是否启用加速，启用为 1，关闭为 0"/>
```

例如针对安兔兔的参考如下：

```
<app package="com.antutu.ABenchMark" mode="1"/>
<app package="com.antutu.benchmark.full" mode="1"/>
```

编译时会将文件打包进固件。

12.18 GPU相关问题排查方法

参考下面文档，可以做初步的问题排查

```
RKDocs/android/Rockchip_User_Guide_Dr.G_CN&EN.pdf
```

12.19 OTP和efuse说明

OTP支持芯片

- RK3326
- PX30
- RK3566
- RK3568
- RK3572
- RK3588

EFUSE支持芯片

- RK3288
- RK3368
- RK3399

固件签名和otp/efuse烧写参考文档

```
RKDocs/common/security/Rockchip-Secure-Boot-Application-Note-V1.9.pdf
```

12.20 代码中如何判断设备的OTP/EFUSE是否已经烧写

OTP/EFUSE的状态会通过kernel的cmdline进行传递，cmdline中的fuse.programmed用来标识OTP/EFUSE状态，具体如下：

- "fuse.programmed=1"：软件固件包已经进行了secure-boot签名，硬件设备的efuse/otp已经被烧写。

- "fuse.programmed=0"：软件固件包已经进行了secure-boot签名，硬件设备的efuse/otp没有被烧写。
- cmdline中没有fuse.programmed：软件固件包没有进行secure-boot签名（Miniloader不传递），或者Miniloader太旧没有支持传递。

12.21 开关selinux

如下修改，false为关闭，true为打开

```
device/rockchip/common$
--- a/BoardConfig.mk
+++ b/BoardConfig.mk
@@ -67,7 +67,7 @@ endif

 # Enable android verified boot 2.0
 BOARD_AVB_ENABLE ?= false
-BOARD_SELINUX_ENFORCING ?= false
+BOARD_SELINUX_ENFORCING ?= true
```

12.22 开机弹出”Android系统出现问题”警告

出现警告框的原因有两种：

1. 固件不匹配，system/boot/vendor三个fingerprint不一致，不是同一套固件。
2. 机器打开支持了IO调试功能的config，编译时，使用文档前面所说的内核编译命令即可关闭。
3. 对于需要使用IO调试功能的项目，可以直接不管上述两种原因，直接合入frameworks/base下的patch去掉弹窗：

```
diff --git
a/services/core/java/com/android/server/wm/ActivityTaskManagerService.java
b/services/core/java/com/android/server/wm/ActivityTaskManagerService.java
index 595c340..d4e495a 100644
--- a/services/core/java/com/android/server/wm/ActivityTaskManagerService.java
+++ b/services/core/java/com/android/server/wm/ActivityTaskManagerService.java
@@ -6555,7 +6555,7 @@ public class ActivityTaskManagerService extends
IActivityTaskManager.Stub {
                 } catch (RemoteException e) {
                 }

-                if (!Build.isBuildConsistent()) {
+                if (0 && !Build.isBuildConsistent()) {
                     Slog.e(TAG, "Build fingerprint is not consistent, warning
user");
                     mUiHandler.post(() -> {
                         if (mShowDialogs) {
```

12.23 如何打开设置中以太网的设置项

系统设置中默认没有以太网设置的选项，如果项目中需要以太网可以按如下配置打开：

```
--- a/BoardConfig.mk
+++ b/BoardConfig.mk
@@ -146,3 +146,6 @@ endif
 ifeq ($(strip $(BOARD_USES_AB_IMAGE)), true)
 DEVICE_MANIFEST_FILE :=
device/rockchip/$(TARGET_BOARD_PLATFORM)/manifest_ab.xml
 endif

+# for ethernet
+BOARD_HS_ETHERNET := true
```

12.24 关于AVB和security boot的操作

AVB和security boot的操作参考文档

```
RKDocs/common/security/Rockchip_Developer_Guide_Android_SecurityBoot_AVB_CN.pdf
```

12.25 IO命令无法使用

IO命令需要依赖DEVMEM，而DEVMEM默认是关闭的，所以导致IO默认无法使用，如果调试需要使用IO命令可以按如下修改：

```
user@ubuntu:~/rk3572/$ vim mkcombinedroot/configs/android-16.config
```

如果是GO的产品则需要修改：

```
user@ubuntu:~/rk3572$ vim mkcombinedroot/configs/android-16-go.config
```

删除掉下面这行：

```
# CONFIG_DEVMEM is not set
```

如果要编译Android，则还需要修改如下代码

```
cd rk3572/kernel-6.12/kernel/configs

diff --git a/android-6.12/android-base.config b/android-6.12/android-base.config
```

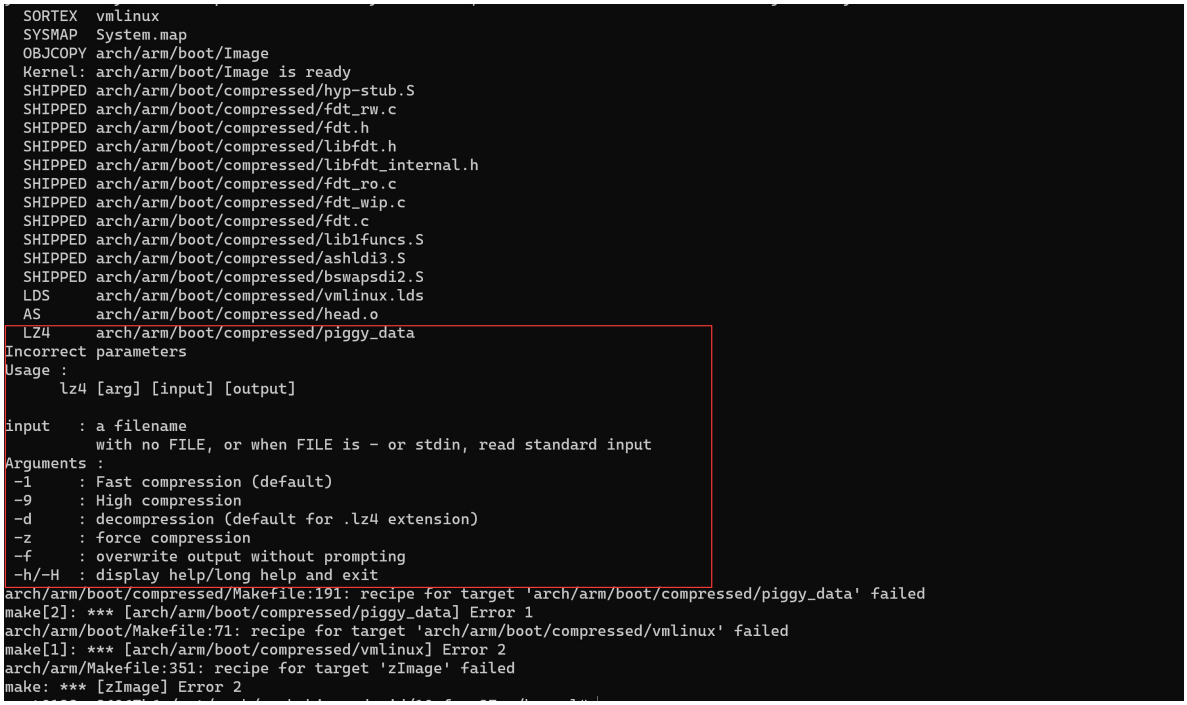
```
index 5de76f0..6dcdf86 100644
--- a/android-6.12/android-base.config
+++ b/android-6.12/android-base.config
@@ -2,7 +2,6 @@
 # CONFIG_ANDROID_LOW_MEMORY_KILLER is not set
 # CONFIG_ANDROID_PARANOID_NETWORK is not set
 # CONFIG_BPFILTER is not set
-# CONFIG_DEVMEM is not set
 # CONFIG_FHANDLE is not set
 # CONFIG_FW_CACHE is not set
 # CONFIG_IP6_NF_NAT is not set
user@host:~/rk3572/kernel/configs$ git diff
diff --git a/u/android-6.12/android-base.config b/u/android-6.12/android-
base.config
index 29b9e98..c1b21cf 100644
--- a/u/android-6.12/android-base.config
+++ b/u/android-6.12/android-base.config
@@ -2,7 +2,6 @@
 # CONFIG_ANDROID_LOW_MEMORY_KILLER is not set
 # CONFIG_ANDROID_PARANOID_NETWORK is not set
 # CONFIG_BPFILTER is not set
-# CONFIG_DEVMEM is not set
 # CONFIG_FHANDLE is not set
 # CONFIG_FW_CACHE is not set
 # CONFIG_IP6_NF_NAT is not set
```

12.26 SN号的命名规则

SN号必须以字母开头，长度14个字节以内。

12.27 Kernel编译报LZ4的错误

Kernel编译的时候报如下错误：



问题原因：

系统自带的lz4版本太低，要求1.8.3及以上版本

```
user@ubuntu:~$ lz4 -v
*** LZ4 command line interface 64-bits v1.8.3, by Yann Collet ***
refusing to read from a console
```

解决方法：
直接拷贝android编译出来的lz4覆盖系统的lz4

```
sudo cp out/host/linux-x86/bin/lz4 /usr/bin/lz4
```

12.28 Android Samba功能

参考文档

```
RKDocs/android/Rockchip_Introduction_Android_Samba_CN.pdf
```

12.29 NFS启动

参考文档及补丁：

```
RKDocs/android/patches/customized_functions/nfs_boot_patch_v1.1.0.zip
```

13. 多屏异显异触

参考文档

```
RKDocs/android/patches/customized_functions/Android异显开发说明.zip
```

14. 多屏异声

参考文档

```
RKDocs/android/patches/customized_functions/Dual_Audio_v1.0.zip
```

15. 附录 A 编译开发环境搭建 Compiling and development environment setup

15.1 Initializing a Build Environment

This section describes how to set up your local work environment to build the Android source files. You must use Linux or Mac OS; building under Windows is not currently supported.
For an overview of the entire code-review and code-update process, see Life of a Patch.
Note: All commands in this site are preceded by a dollar sign ($) to differentiate them from output or entries within files. You may use the Click to copy feature at the top right of each command box to copy all lines without the dollar signs or triple-click each line to copy it individually without the dollar sign.

15.2 Choosing a Branch

Some requirements for the build environment are determined by the version of the source code you plan to compile. For a full list of available branches, see Build Numbers. You can also choose to download and build the latest source code (called master), in which case you will simply omit the branch specification when you initialize the repository.
After you have selected a branch, follow the appropriate instructions below to set up your build environment.

15.3 Setting up a Linux build environment

These instructions apply to all branches, including master.
The Android build is routinely tested in house on recent versions of Ubuntu LTS (14.04) and Debian testing. Most other distributions should have the required build tools available.
For Gingerbread (2.3.x) and newer versions, including the master branch, a 64-bit environment is required. Older versions can be compiled on 32-bit systems.
Note: See Requirements for the complete list of hardware and software requirements, then follow the detailed instructions for Ubuntu and Mac OS below.

15.4 Installing the JDK

The master branch of Android in the Android Open Source Project (AOSP) comes with prebuilt versions of OpenJDK below prebuilts/jdk/ so no additional installation is required.
Older versions of Android require a separate installation of the JDK. On Ubuntu, use OpenJDK. See JDK Requirements for precise versions and the sections below for instructions.
For Ubuntu >= 15.04
Run the following:

```
sudo apt-get update
sudo apt-get install openjdk-8-jdk
```

For Ubuntu LTS 14.04
There are no available supported OpenJDK 8 packages for Ubuntu 14.04. The Ubuntu 15.04 OpenJDK 8 packages have been used successfully with Ubuntu 14.04. Newer package versions (e.g. those for 15.10, 16.04) were found not to work on 14.04 using the instructions below.

1. Download the .deb packages for 64-bit architecture from old-releases.ubuntu.com:

```
openjdk-8-jre-headless_8u45-b14-1_amd64.deb with SHA256
0f5aba8db39088283b51e00054813063173a4d8809f70033976f83e214ab56c0
openjdk-8-jre_8u45-b14-1_amd64.deb with SHA256
9ef76c4562d39432b69baf6c18f199707c5c56a5b4566847df908b7d74e15849
openjdk-8-jdk_8u45-b14-1_amd64.deb with SHA256
6e47215cf6205aa829e6a0a64985075bd29d1f428a4006a80c9db371c2fc3c4c
```

2. Optionally, confirm the checksums of the downloaded files against the SHA256 string listed with each package above. For example, with the sha256sum tool:

```
    sha256sum {downloaded.deb file}
```

3. Install the packages:

```
sudo apt-get update
```

Run dpkg for each of the .deb files you downloaded. It may produce errors due to missing dependencies:

```
sudo dpkg -i {downloaded.deb file}
```

To fix missing dependencies:

```
sudo apt-get -f install
```

Update the default Java version - optional
Optionally, for the Ubuntu versions above update the default Java version by running:

```
sudo update-alternatives --config java
sudo update-alternatives --config javac
```

Note: If, during a build, you encounter version errors for Java, see Wrong Java version for likely causes and solutions.
Installing required packages (Ubuntu 14.04)
You will need a 64-bit version of Ubuntu. Ubuntu 14.04 is recommended.

```
sudo apt-get install git-core gnupg flex bison gperf build-essential zip curl
zlib1g-dev gcc-multilib g++-multilib libc6-dev-i386 lib32ncurses5-dev x11proto-
core-dev libx11-dev lib32z-dev ccache libgl1-mesa-dev libxml2-utils xsltproc
unzip python-pyelftools python3-pyelftools device-tree-compiler libfdt-dev
libfdt1 libssl-dev liblz4-tool python-dev
```

Note: To use SELinux tools for policy analysis, also install the python-networkx package. Note: If you are using LDAP and want to run ART host tests, also install the libnss-sss:i386 package.

15.5 Configuring USB Access

Under GNU/linux systems (and specifically under Ubuntu systems), regular users can't directly access USB devices by default. The system needs to be configured to allow such access.
The recommended approach is to create a file /etc/udev/rules.d/51-android.rules (as the root user) and to copy the following lines in it. must be replaced by the actual username of the user who is authorized to access the phones over USB.

```
# adb protocol on passion (Rockchip products)
SUBSYSTEM=="usb", ATTR{idVendor}=="2207", ATTR{idProduct}=="0010", MODE="0600",
OWNER="<username>"
```

Those new rules take effect the next time a device is plugged in. It might therefore be necessary to unplug the device and plug it back into the computer.
This is known to work on both Ubuntu Hardy Heron (8.04.x LTS) and Lucid Lynx (10.04.x LTS). Other versions of Ubuntu or other variants of GNU/linux might require different configurations.
References : http://source.android.com/source/initializing.html

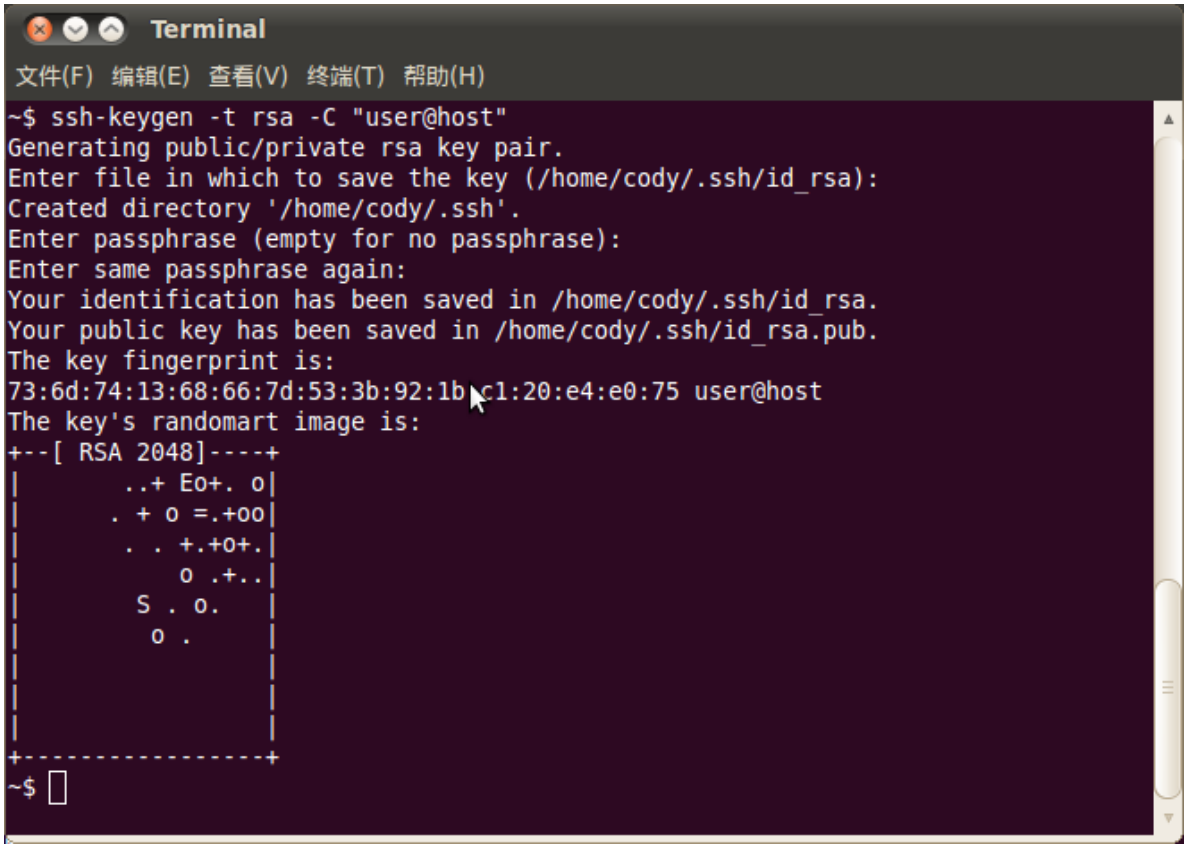
16. 附录 B SSH公钥操作说明 SSH public key operation instruction

16.1 附录 B-1 SSH公钥生成 SSH public key generation

使用如下命令生成：

```
ssh-keygen -t rsa -C "user@host"
```

请将user@host替换成您的邮箱地址。



命令运行完成会在你的目录下生成key文件。

```
~$ ls -l .ssh/
总用量 8
-rw------- 1 cody cody 1675 2012-10-15 11:38 id_rsa
-rw-r--r-- 1 cody cody  391 2012-10-15 11:38 id_rsa.pub
```

请妥善保存生成的私钥文件id_rsa和密码，并将id_rsa.pub发邮件给SDK发布服务器的管理员。

16.2 附录 B-2 使用key-chain管理密钥 Use key-chain to manage the key

推荐您使用比较简易的工具keychain管理密钥。
具体使用方法如下：

1. 安装keychain软件包：

```
$sudo aptitude install keychain
```

2. 配置使用密钥：

```
$vim ~/.bashrc
```

增加下面这行：

```
eval `keychain --eval ~/.ssh/id_rsa`
```

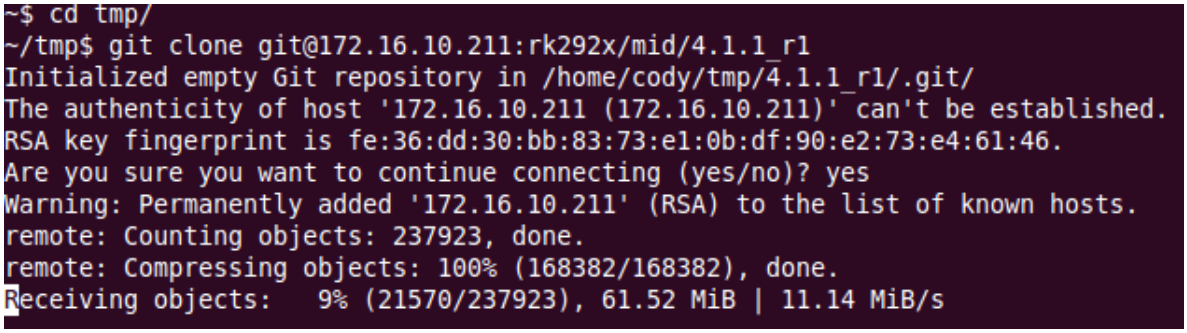
其中，id_rsa是私钥文件名称。
以上配置以后，重新登录控制台，会提示输入密码，只需输入生成密钥时使用的密码即可，若无密码可不输入。
另外，请尽量不要使用sudo或root用户，除非您知道如何处理，否则将导致权限以及密钥管理混乱。

16.3 附录 B-3 多台机器使用相同ssh公钥 Multiple devices use the same ssh public key

在不同机器使用，可以将你的ssh私钥文件id_rsa拷贝到要使用的机器的“~/.ssh/id_rsa”即可。
在使用错误的私钥会出现如下提示，请注意替换成正确的私钥。

```
~/tmp$ git clone git@172.16.10.211:rk292x/mid/4.1.1_r1
Initialized empty Git repository in /home/cody/tmp/4.1.1_r1/.git/
The authenticity of host '172.16.10.211 (172.16.10.211)' can't be established.
RSA key fingerprint is fe:36:dd:30:bb:83:73:e1:0b:df:90:e2:73:e4:61:46.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '172.16.10.211' (RSA) to the list of known hosts.
git@172.16.10.211's password:
```

添加正确的私钥后，就可以使用git 克隆代码，如下图。



添加ssh私钥可能出现如下提示错误。

```
Agent admitted failure to sign using the key
```

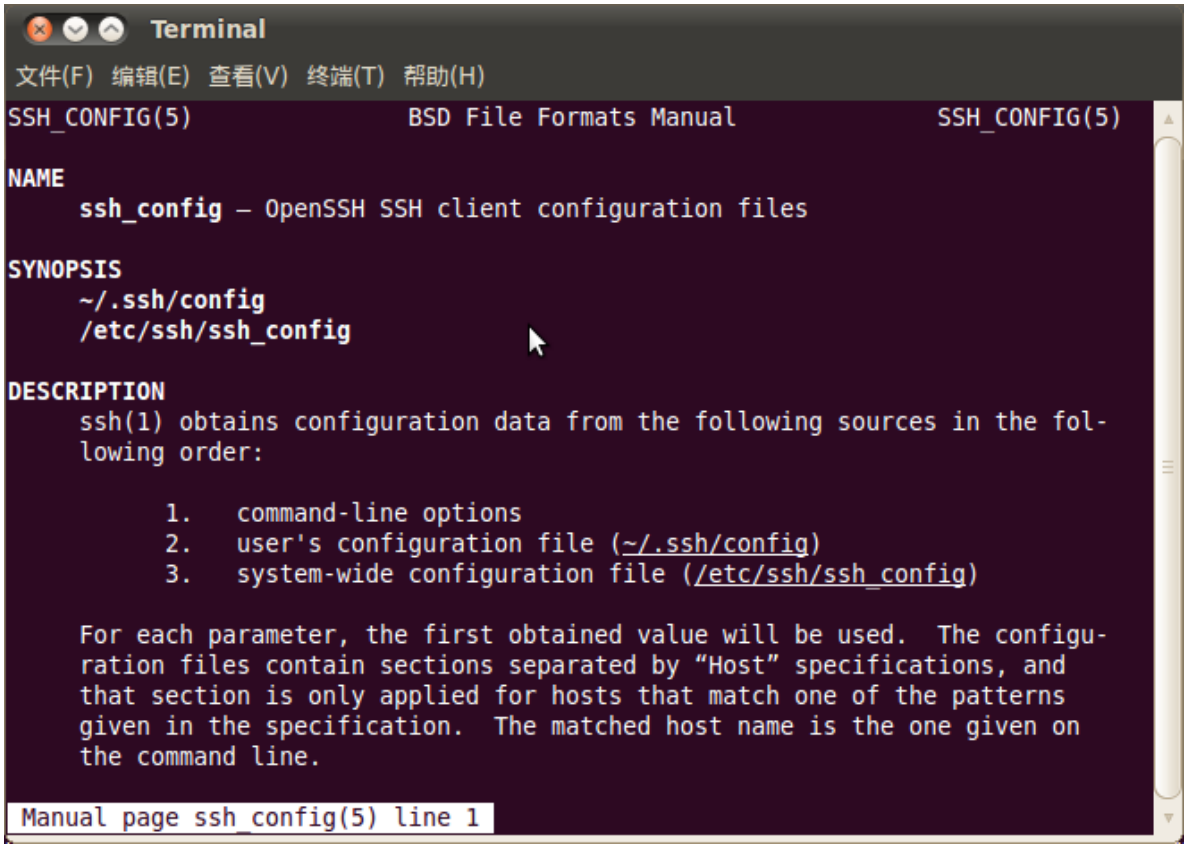
在console输入如下命令即可解决。

```
ssh-add ~/.ssh/id_rsa
```

16.4 附录 B-4 一台机器切换不同ssh公钥 Switch different ssh public keys on one device

可以参考ssh_config文档配置ssh。

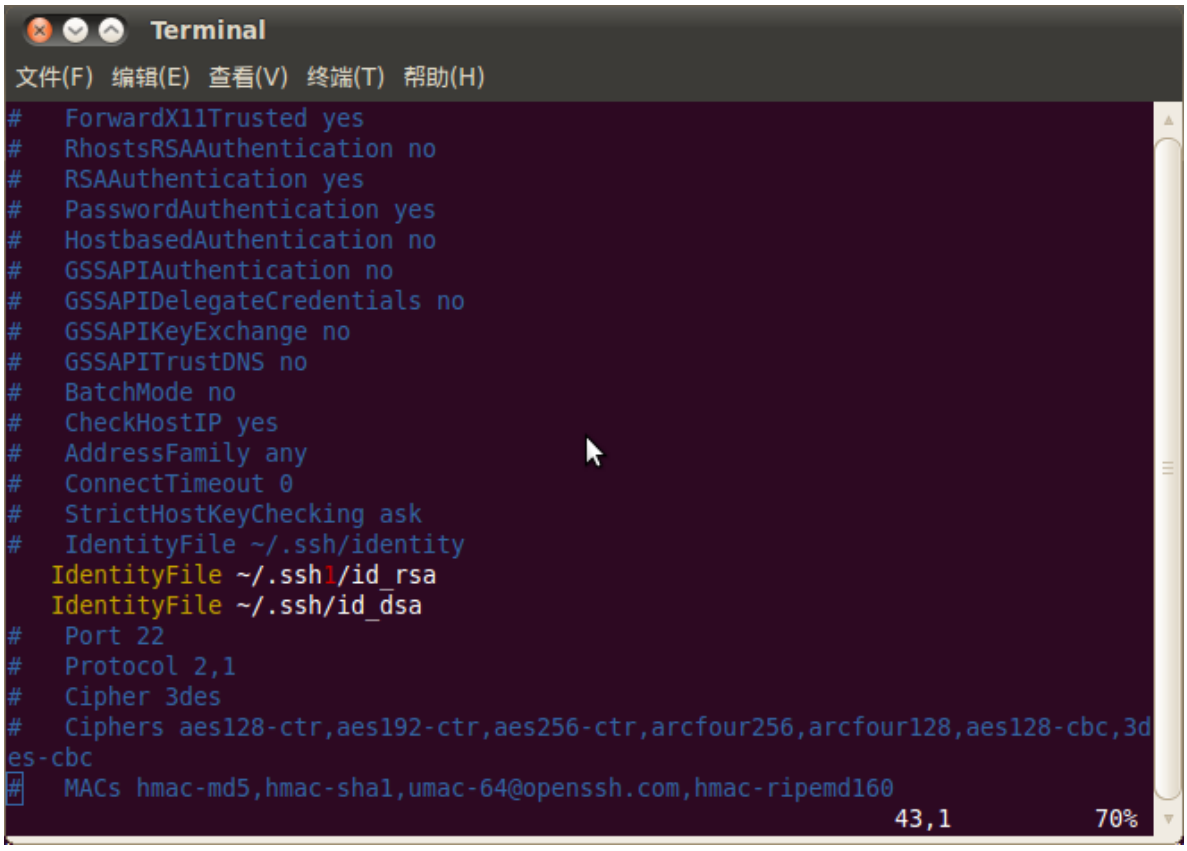
```
~$ man ssh_config
```



通过如下命令，配置当前用户的ssh配置。

```
~$ cp /etc/ssh/ssh_config ~/.ssh/config
~$ vi .ssh/config
```

如图，将ssh使用另一个目录的文件“~/.ssh1/id_rsa”作为认证私钥。通过这种方法，可以切换不同的密钥。



16.5 附录 B-5 密钥权限管理 Key authority management

服务器可以实时监控某个key的下载次数、IP等信息，如果发现异常将禁用相应的key的下载权限。
请妥善保管私钥文件。并不要二次授权与第三方使用。

16.6 附录 B-6 Git权限申请说明 Git authority application instruction

参考上述章节，生成公钥文件，发邮件至 fae@rock-chips.com，申请开通SDK代码下载权限。